

海洋结构分析通用软件 SAM

二次开发手册

V4.0

20231212



开发单位： 中国船舶科学研究中心及下属中船奥蓝托无锡软件技术有限公司

地址： 江苏省无锡市滨湖区山水东路 222 号

网址： www.cssrc.com.cn

目 录

1 简介	1
2 Python 二次开发	2
2.1 二次开发简介	2
2.2 脚本二次开发	2
2.2.1 脚本编写	2
2.2.2 脚本执行	4
2.2.3 实例 1: 参数化创建加筋板前处理建模	14
2.2.4 实例 2: 参数化加筋板计算及后处理	18
2.3 界面二次开发	20
2.3.1 启动项	20
2.3.2 通过脚本初始化需要加载的模块	20
2.3.3 模块	22
3 C++二次开发	30
3.1 工程构建及环境配置	30
3.1.1 构建主目录	31
3.1.2 配置 SAM 开发环境	31
3.1.3 构建工程目录	32
3.1.4 管理工程目录	33
3.1.5 新建项目及项目管理	34
3.1.6 添加代码文件	37
3.1.7 使用 cmake 构建工程	45
3.2 生成动态库及使用	48
3.2.1 动态库生成	48
3.2.2 使用动态库	48
3.1 C++函数说明	53
3.1.1 共性基础类	53
3.1.2 Mdb	56
3.1.3 Model	57
3.1.4 Part	59
3.1.5 Material	61
3.1.6 Profile	62
3.1.7 Section	64
3.1.8 Assignment	66
3.1.9 Assembly	66
3.1.10 Step	67
3.1.11 BC	71
3.1.12 Loads	72
3.1.13 Mesh	75
3.1.14 基础数据类	53
3.1.15 渲染更新	77
4 输入输出格式说明	77
4.1 输入格式关键字	77
4.1.1 *Heading	77
4.1.2 *Part	77

4.1.3 *Node	78
4.1.4 *Element.....	78
4.1.5 *Nset.....	79
4.1.6 *Elset.....	79
4.1.7 *Chief	80
4.1.8 *Assembly	80
4.1.9 *Instance.....	80
4.1.10 *Submodel.....	81
4.1.11 *Beam Section.....	81
4.1.12 *Shell Section.....	82
4.1.13 *Material	83
4.1.14 *Density	83
4.1.15 *Acoustic Medium	83
4.1.16 *Amplitude.....	84
4.1.17 *Step.....	84
4.1.18 *Boundary	85
4.1.19 *ALoad.....	86
5 输出格式说明	87
5.1 模型信息.....	87
5.1.1 Group: Parts.....	87
5.1.2 Group: Assembly.....	89
5.2 结果信息.....	92
5.2.1 Group: Steps	92

1 简介

海洋结构分析通用软件(SAM), 以结构有限元分析为核心, 对标 Nastran、Ansys、Abaqus 等商业软件, 支持静力、模态、稳态、瞬态、非线性等常用分析, 精度和商软误差<0.1%, 效率和商软相当。具有自主化、高精度、专业化、全面性、开放性和可靠性等特色。

当前版本使用说明

- 当前版本有限元分析支持静力分析（包括线性，材料非线性，几何非线性），模态分析（包括结构和声学模态分析），稳态动力学分析（即谐响应分析，包括模态叠加法和直接法），响应谱分析，瞬态动力学分析。SAM 整体分析功能和商软 Nastran、Ansys、Abaqus 对比如下：

分析类型		商软				SAM
		重要度	MSC.Nastran	Ansys	Abaqus	
1. 静力分析	通用线性静力	5	√	√	√	√
	线性屈曲分析	2	√	√	√	×
	惯性释放	2	√	√	√	√
2. 模态分析	模态分析	5	√	√	√	√
3. 稳态响应分析	正弦响应分析	4	√	√	√	√
	响应谱分析	4	√	√	√	√
	随机响应分析	2	√	√	√	×
4. 瞬态动力学	隐式动力学	3	√	√	√	√
	显式动力学	4	×	×	√	×
5. 非线性分析	材料非线性	5	√	√	√	√
	几何非线性	3	√	√	√	√
	边界非线性	3	×	√	√	×

- 目前，SAM 有限元求解器的精度和商软 CAE 一致，SAM 包括 Abaqus 和 Nastran 单元，其中 SAM 采用 Abaqus 单元时所有模型线性/非线性精度和 Abaqus 的误差<0.1%, SAM 采用 Nastran 单元时所有模型线性精度和 Nastran 误差<1%。同时，由于各个有限元求解器的算法不同，不同软件之间的计算结果会有偏差，默认情况下，SAM 采用 Abaqus 单元，此时可能会和 Nastran

有差异。想要 SAM 和 Nastran 的分析结果更加接近，那么需要在 SAM 中改为 Nastran 单元。

- 当前版本只支持简单的几何体创建功能，对复杂模型，均采用导入 Abaqus 的 *.inp 或者 Nastran 的 bdf 的有限元模型的方式，如原有模型采用 Ansys 的 cdb 等模型，需要通过 Abaqus 转成 inp 后再导入到 SAM 中。
- 单位制：SAM 无单位制的限定，对所有单位制都可执行，只要各种物理量满足同一个单位制就行，譬如如果用 SI（mm），那么几何尺寸就是 mm，而杨氏模量用 MPa。

2 Python 二次开发

2.1 二次开发简介

软件提供了 Python 扩展接口，开发人员可进行命令行脚本和界面的二次开发。

2.2 脚本二次开发

软件采用 Python 进行脚本二次开发，可用于命令行脚本到创建定制的应用程序。

如需调用数学库，需导入 numpy 库，譬如在 Command 窗口采用 log 函数：

```
from numpy import *  
log (10)
```

2.2.1 脚本编写

SAM 界面上的所有操作都记录在了 Command History 窗口。

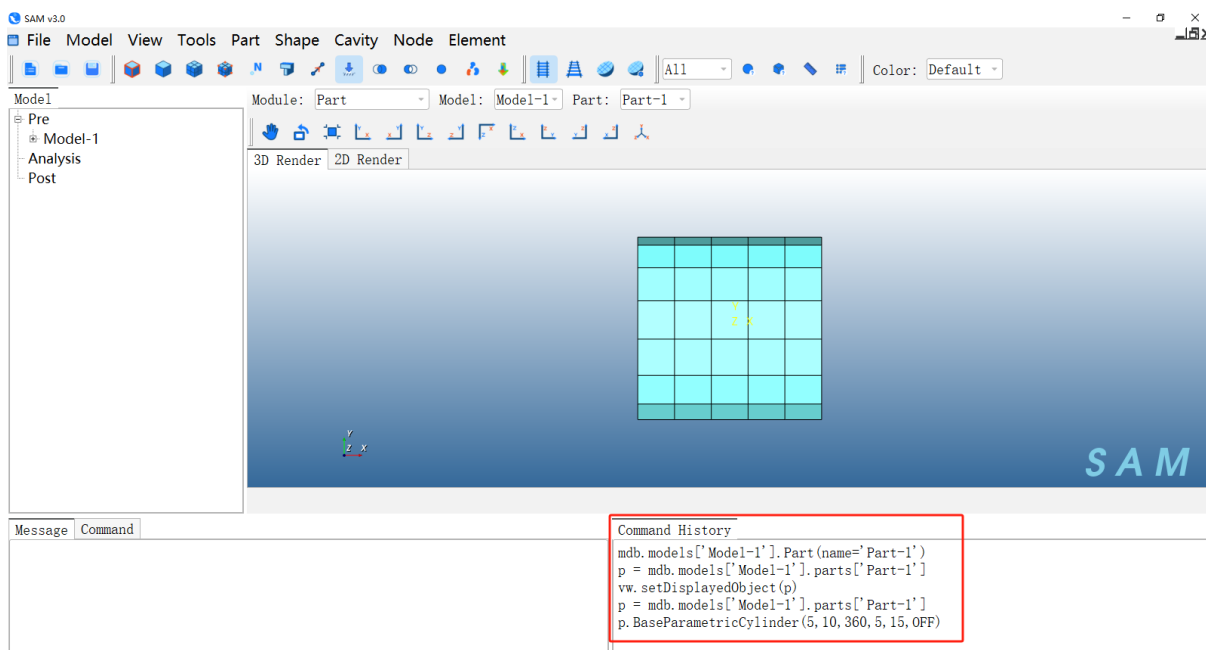


图 1 Command History 窗口显示操作记录

在 Command History 窗口中拷贝该脚本命令，然后新建.py 格式的文件，将脚本命令粘贴进去，即为一个自动化脚本文件。

✧ 脚本可根据实际需要在文本编辑器中修改

2.2.1.1 脚本段落：引用

脚本使用时按需引用 SAM 或第三方库的模块，如：

```
from sam import *
```

```
from samConstants import *
```

表示引用 SAM 的执行模块，其中定义了一些 SAM 常用变量和参数，在使用无界面执行脚本操作时需要引用。

2.2.1.2 脚本段落：命令语句

脚本使用时的命令语句，用来发送操作指令。

```

#定义模型变量
model=mdb.models['Model-1']
#创建Part
mdb.models['Model-1'].Part('Cylinder-1')
#定义部件变量
p = mdb.models['Model-1'].parts['Cylinder-1']
#参数化建模：圆柱壳
p.BaseParametricCylinder(5,10,360,5,40,OFF)
#定义装配体变量
assembly=model.rootAssembly
#创建实例
instance=assembly.Instance(name='Cylinder-1-1',part=p)
#创建静力分析步
model.StaticStep(name='Step-1',previous='Initial',timePeriod=1,initialInc=1,minInc=1E-005,maxInc=1)
#创建材料
material=model.Material(name='Material-1')
#材料密度
material.Density(table=((7800.0,)),)
#材料弹性属性
material.Elastic(table=((2.1e+11,0.3)),)
#创建壳单元截面
model.HomogeneousShellSection(name='Section-1',material='Material-1',thickness=0.001)

```

图 2 部分脚本命令

2.2.2 脚本执行

2.2.2.1 SAM 界面执行自动化脚本

打开 SAM。

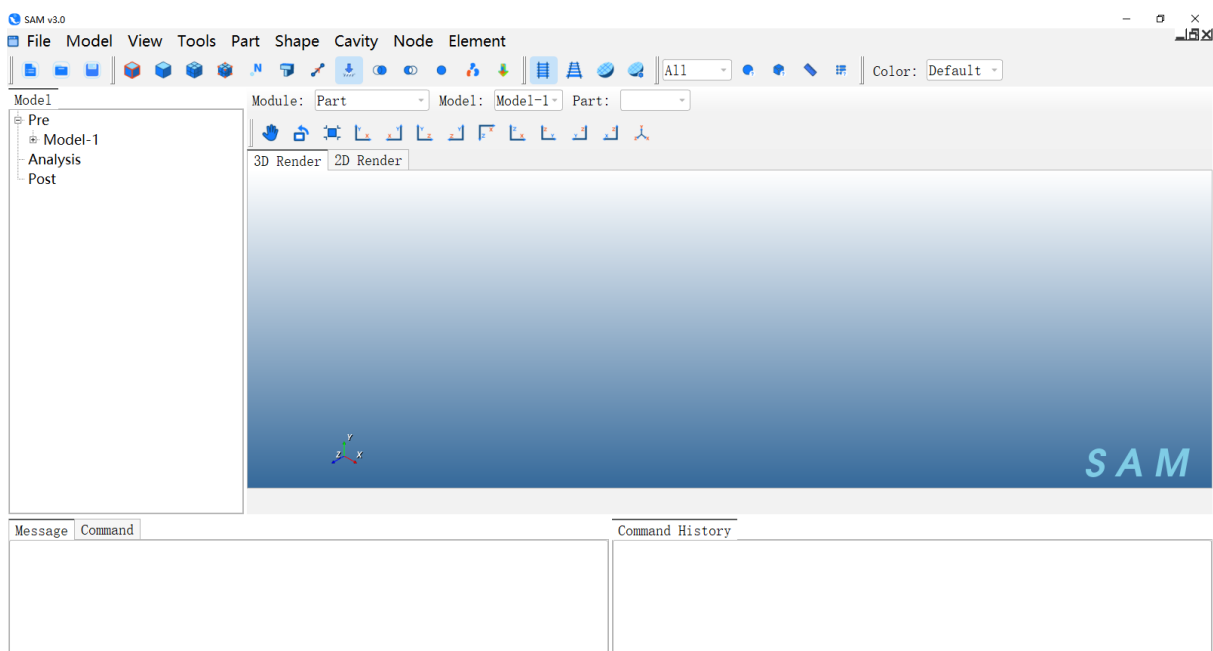


图 3 打开 SAM 界面

在 SAM 软件中点击菜单 **File→Run Script...**。

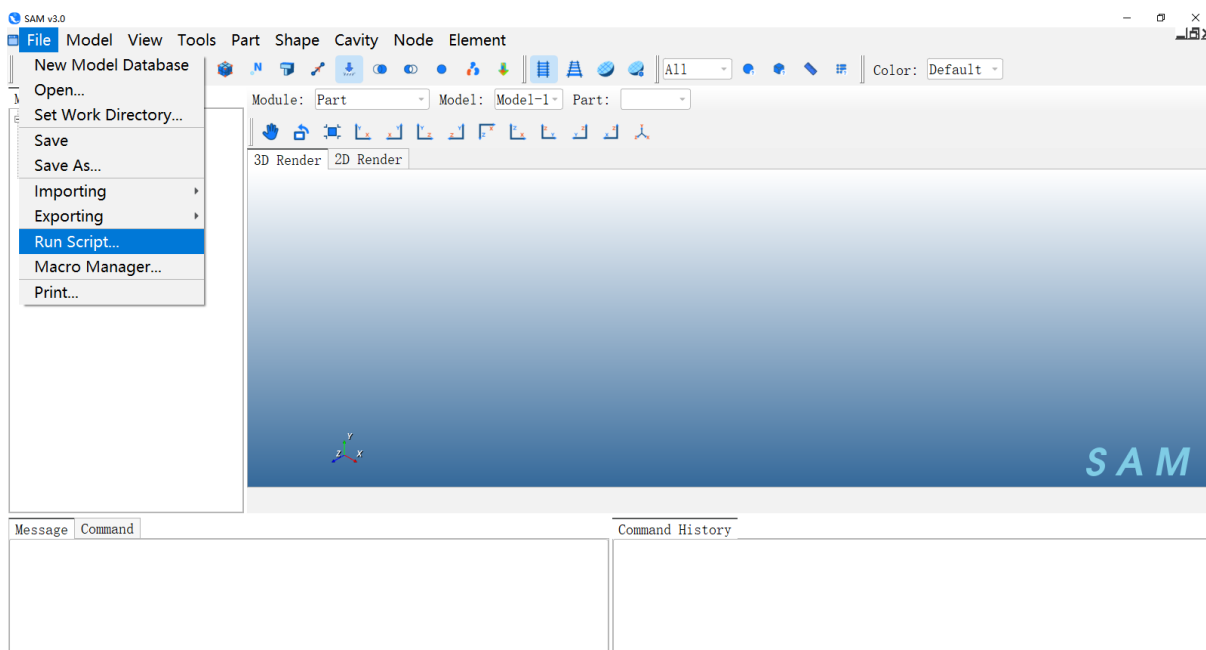


图 4 导入脚本命令

选择脚本文件，譬如安装目录下\Example\Scripts\StaticAnalysis.py

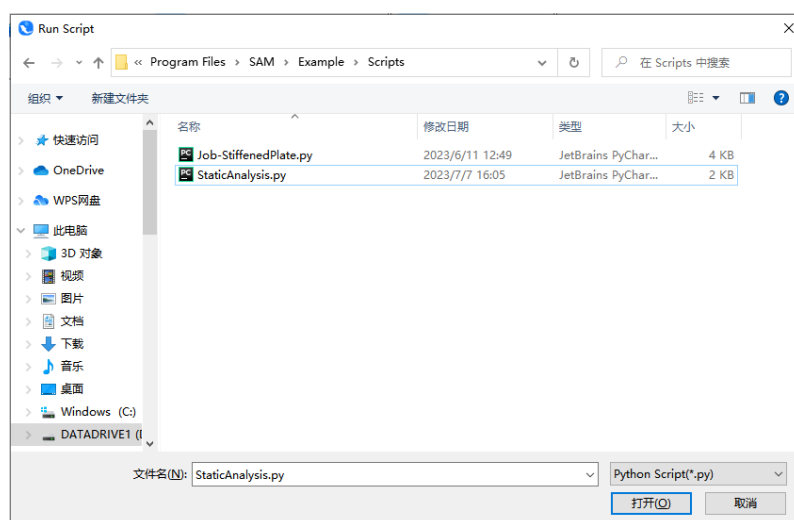


图 5 选择脚本命令文件

如下图所示，自动生成 python 脚本命令流实现模型自动创建

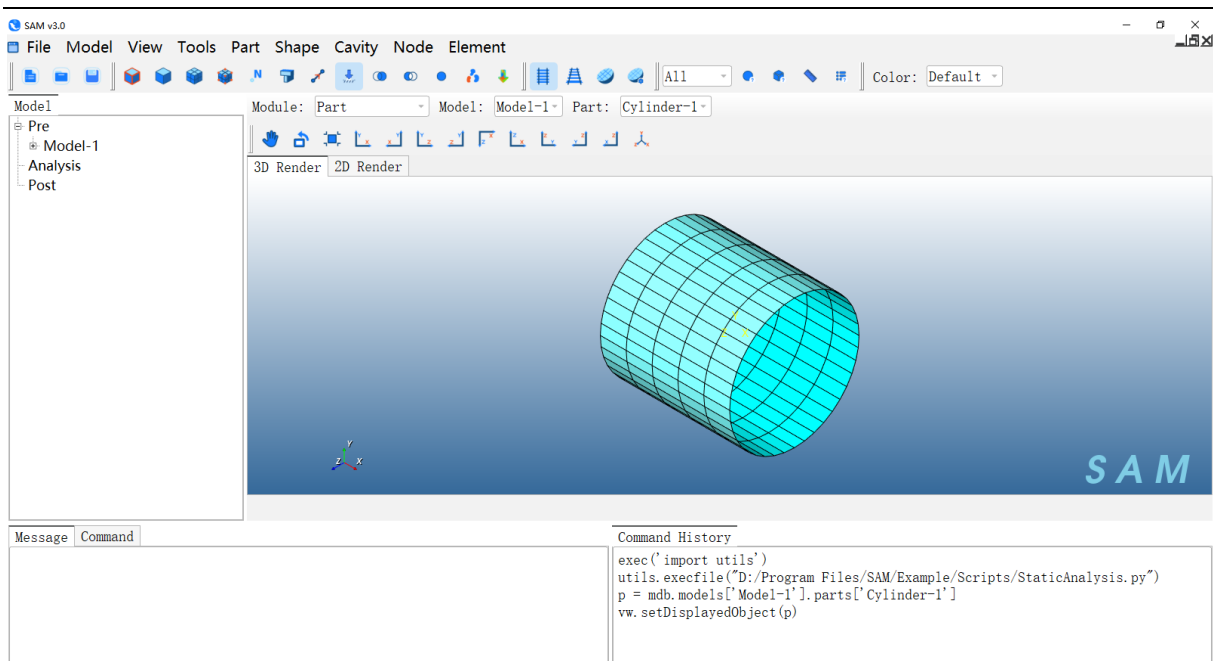


图 6 使用脚本命令实现模型自动创建

切换 Module 至 Job 模块，点击菜单 **Job→Manager**，可发现已经有一个 Job。

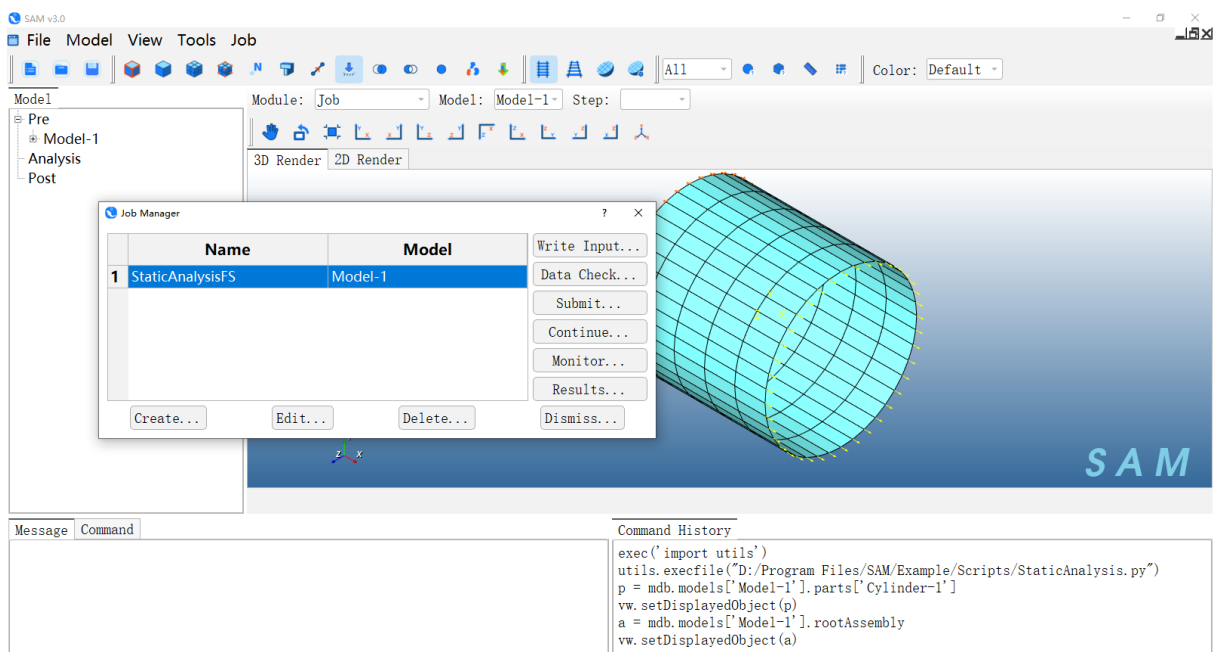


图 7 Job Manager

可直接点击 **Submit** 进行提交任务计算及后处理分析等后续操作。

2.2.2.2 无界面执行自动化脚本

SAM 支持通过命令行不打开界面直接后台执行创建的自动化脚本。如下图所示，在系统的命令行窗口按规定格式输入语句，即可后台执行自动化脚本。

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [版本 10.0.19044.2728]
(c) Microsoft Corporation。保留所有权利。

C:\Users\15861>sam.bat script StaticAnalysis.py
```

图 8 命令执行方式

命令语句格式为：**sam.bat script [script]**

其中，**sam.bat** 为固定项，表示软件启动的批处理文件；

script 为固定项，表示软件启动时为 **python** 模式；

[script]为修改项，用来指定运行的脚本文件名称。

- ✧ 当需要运行的脚本文件不在当前目录时，需要输入脚本文件完整的路径。
- ✧ 后台运行的自动化脚本中不能包含对渲染模型的操作，比如 **setDisplayObject**

拷贝安装目录下 **\Example\Scripts\StaticAnalysis.py** 到某个单独目录（譬如 **D:\temp**）下

2.2.2.2.1 第一次运行脚本

1) 任意目录打开命令行， 运行

sam.bat script StaticAnalysis.py（脚本所在目录）

sam.bat script D:\\temp\\StaticAnalysis.py（其他目录则需要指定脚本所在路径）

此时，**SAM** 会自动创建圆柱模型，在当前目录生成输出 **inp** 文件 **StaticAnalysisFS.inp**。

2) 可以用文本编辑器打开结果文件，里面记录了模型的节点、单元、材料属性、分析步、约束、载荷等信息。

3) 可以打开 **SAM** 查看模型信息，点击 **File→Importing→Model**

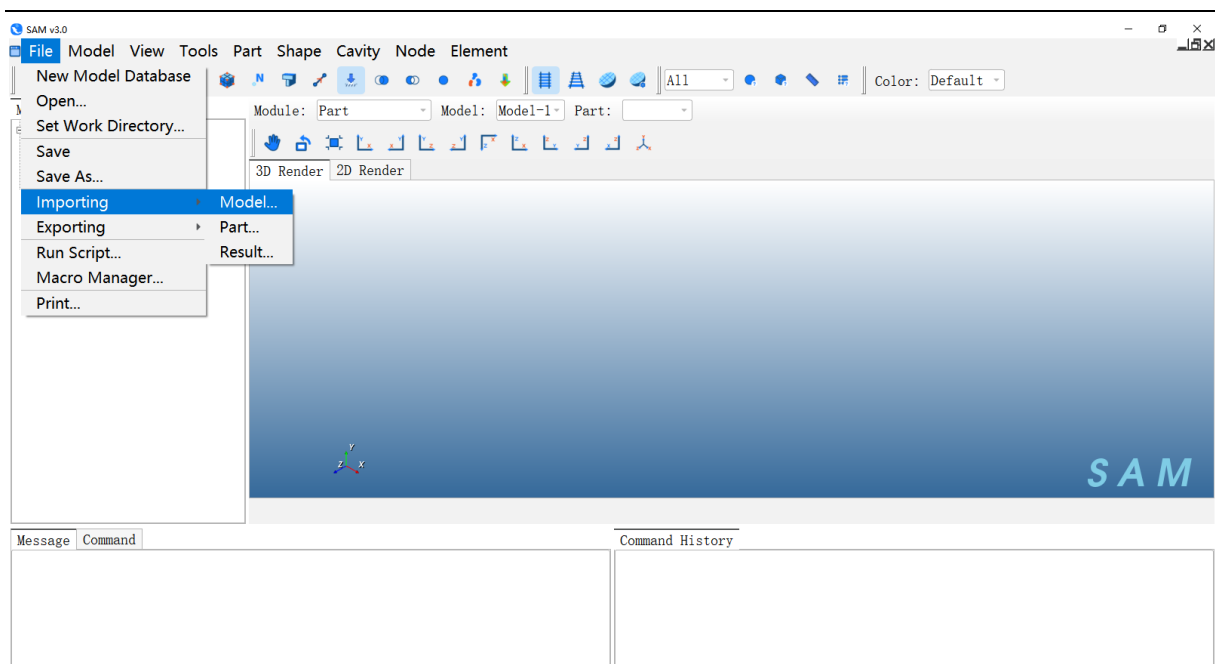


图 9 SAM 打开文件

选取 StaticAnalysisFS.inp 结果文件

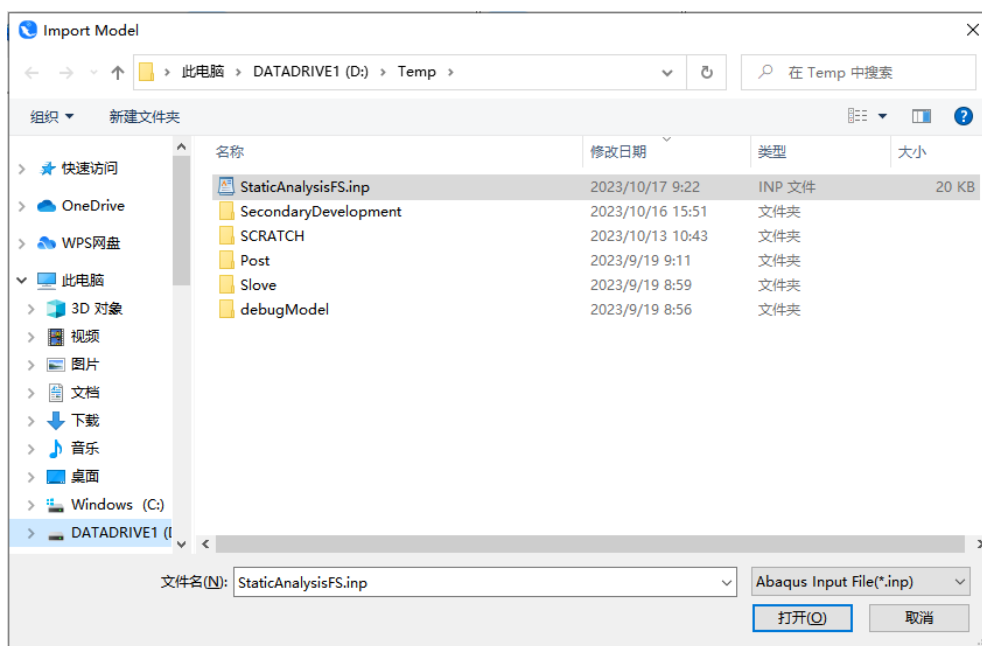


图 10 SAM 导入.inp 文件

导入.inp 查看模型。

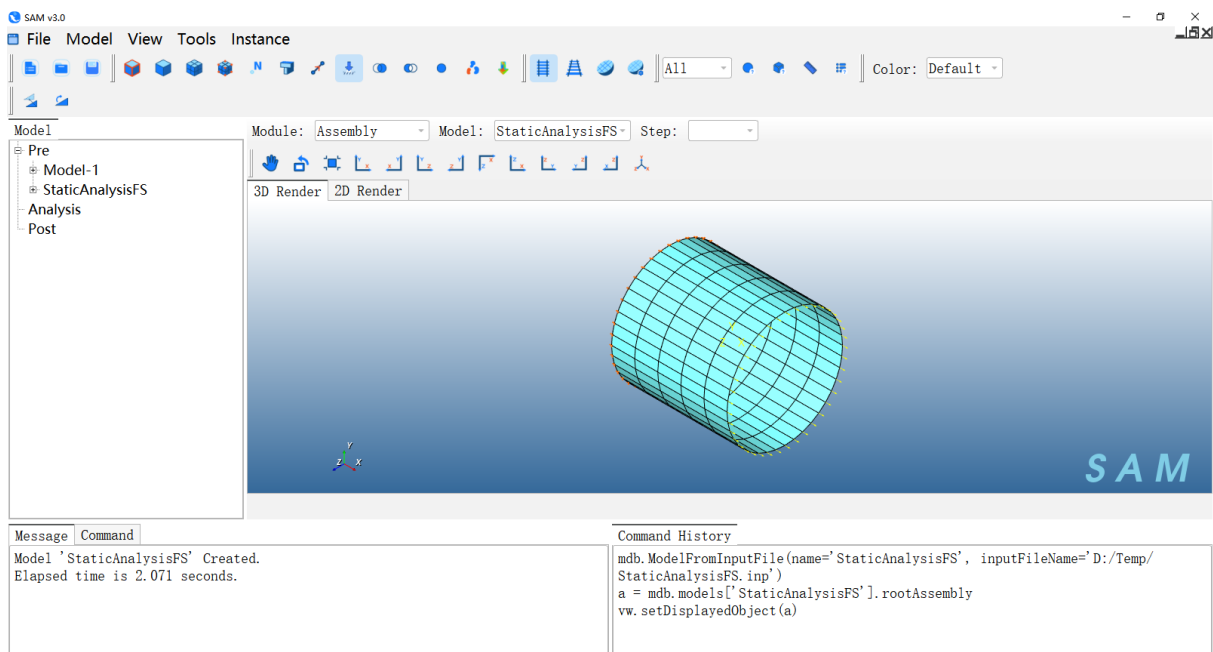


图 11 第一次运行脚本后的结果

2.2.2.2.2 第二次运行脚本

关闭 SAM，在文本编辑器中修改脚本 StaticAnalysis.py，比如把壳单元的厚度从 0.001 改成 0.002，

```
Job-singleSphere.py x StaticAnalysis.py x StaticAnalysis.py x
7 assembly=model.rootAssembly
8 instance=assembly.Instance(model.parts["Cylinder-1"])
9
10 #session.ballValue()
11
12 model.StaticStep("Step-1", "Initial")
13
14 # Create Material
15 material=model.createMaterial("Material-1")
16 material.Elastic([[float(2.1e11), float(0.3)],])
17 material.Density([[float(7800),],])
18
19 model.HomogeneousShellSection("Section-1", "Material-1", 1, 0.002)
20 mdb.models["Model-1"].parts["Cylinder-1"].createSet('Set-Section', 2, [i for i in range(165)])
21 mdb.models["Model-1"].parts["Cylinder-1"].SectionAssignment('Section-1', 'Set-Section')
22 # Create BC
23 assembly.createSetByUserWithInstance('Set-BC', 1, ['Cylinder-1-1'], [[0, 1, 2, 3, 11, 12, 13, 21, 22, 23, 31, 32, 33, 41,
24 setSelected=model.rootAssembly.sets["Set-BC"]
25 model.createBC("BC-1", "Step-1", setSelected, 1)
26 assembly.createSetByUserWithInstance('Set-Load', 1, ['Cylinder-1-1'], [[8, 9, 10, 18, 19, 20, 28, 29, 30, 38, 39, 40, 48
27 setSelected2=model.rootAssembly.sets["Set-Load"]
28 model.createLoad("Load-1", "Step-1", setSelected2, 1, ["0", "1", "1"])
29
30 # Create Job
```

图 12 修改脚本

然后按上述方式再次运行该脚本，重新打开 SAM，查看.inp 文件。Module 切换到 Profile，点击 Section 下的 Manager。

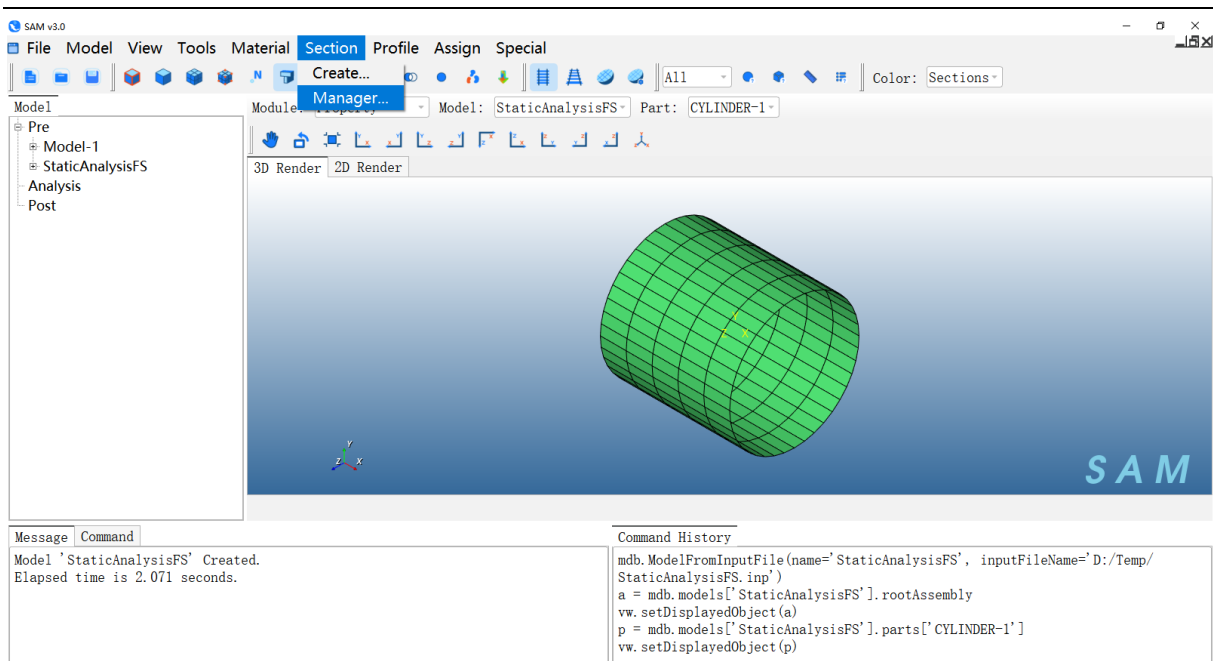


图 13 第二次运行脚本后的模型

在 Manager 窗口中选择 Section-1，点击 Edit 功能按钮，可以看到壳属性的厚度变成了 0.002

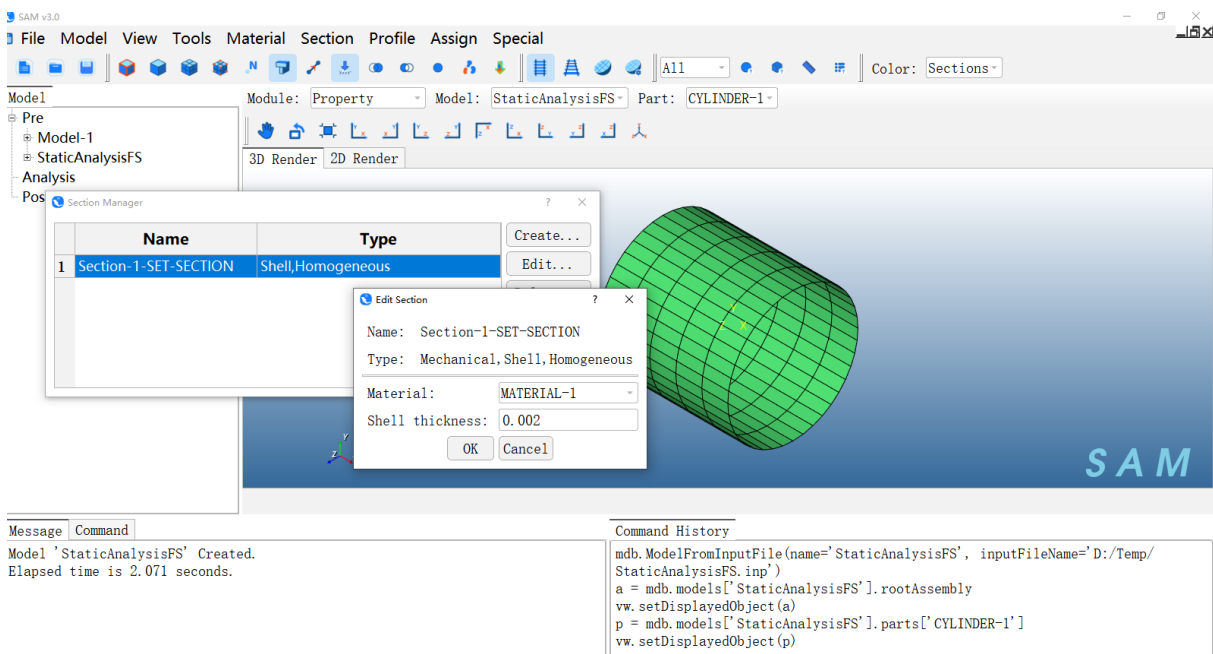


图 14 查看厚度数据

2.2.2.3 SAM 界面分步执行脚本命令

SAM 支持在界面上 Command 窗口通过命令进行建模及仿真计算。以创建材料为例：

打开 SAM，命令窗口位于左下方：

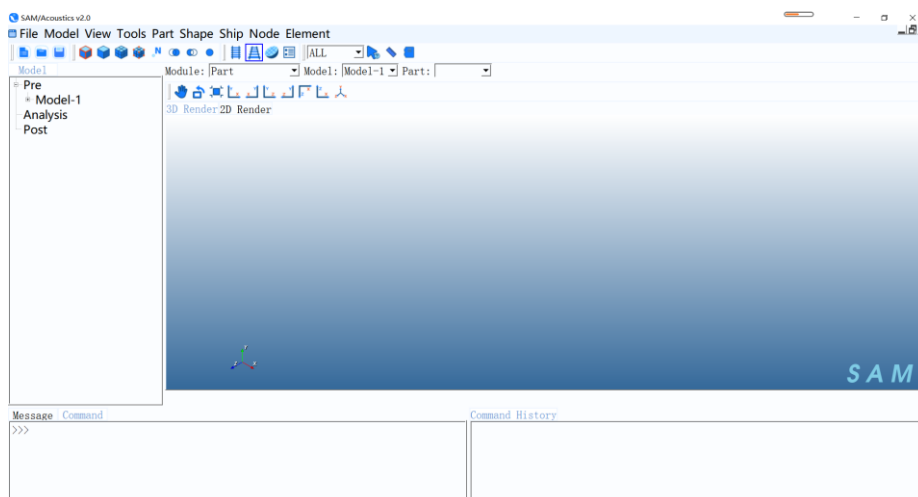


图 15 使用脚本命令实现模型创建以及网格划分

输入创建新材料的命令后，回车：

`mdb.models['Model-1'].Material('MaterialByPython')`

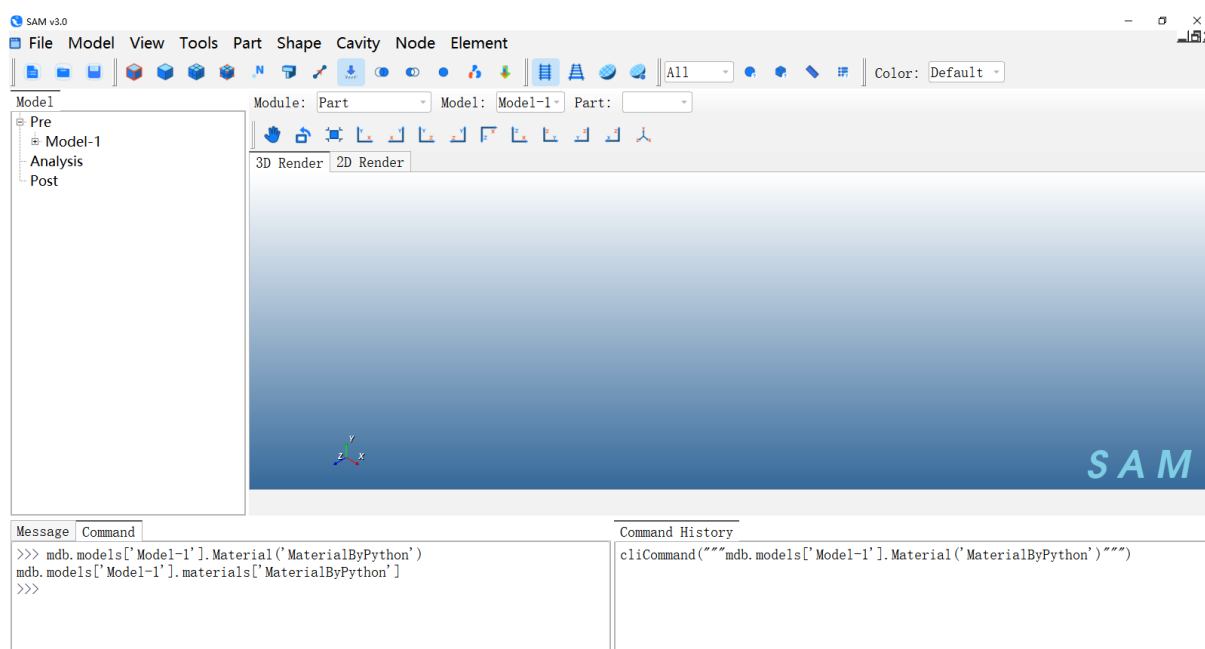


图 16 输入命令

点击 **Module**，切换到 **Property** 模块。点击菜单 **Material**→**Material Manager**。

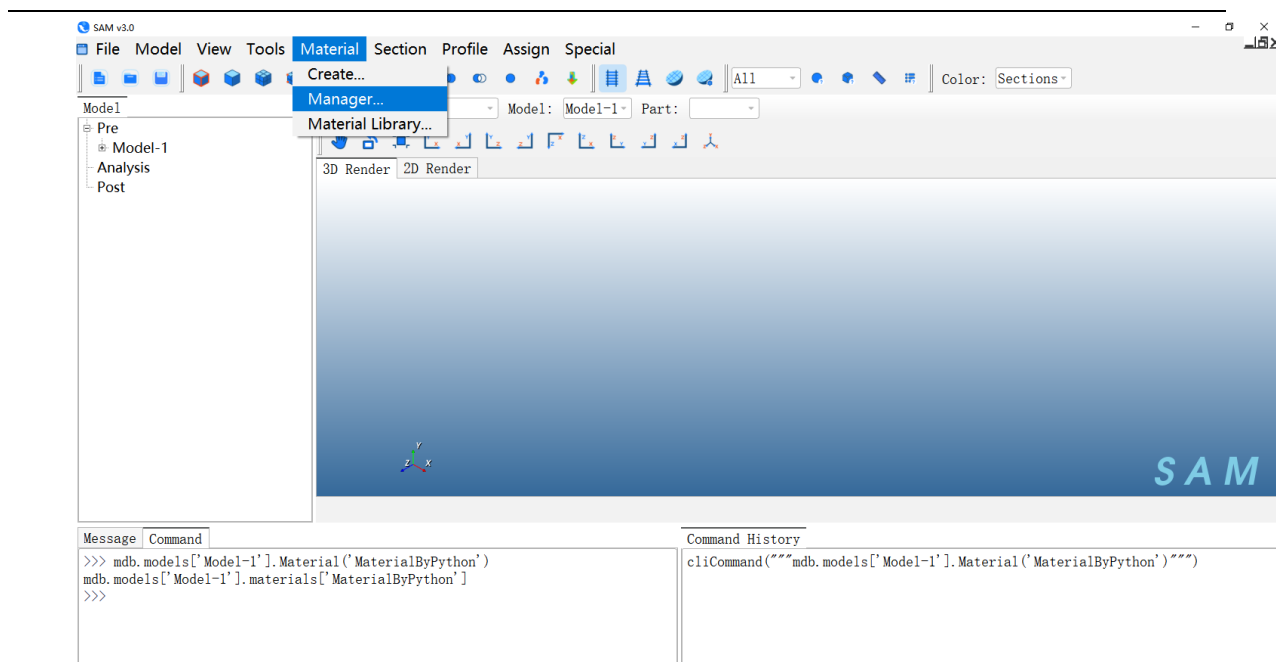


图 17 查看材料

在 Manager 窗口中能够查看到名为“MaterialByPython”的材料名。

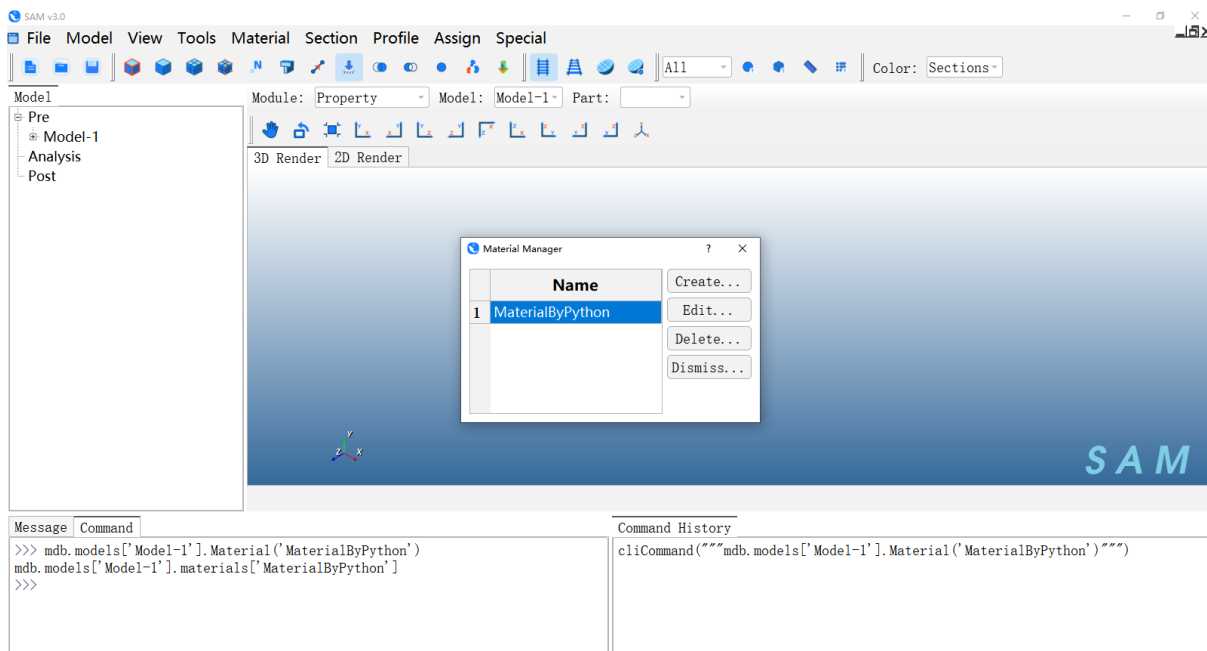


图 18 查看脚本创建的材料

点击 Edit，可发现材料参数为空。

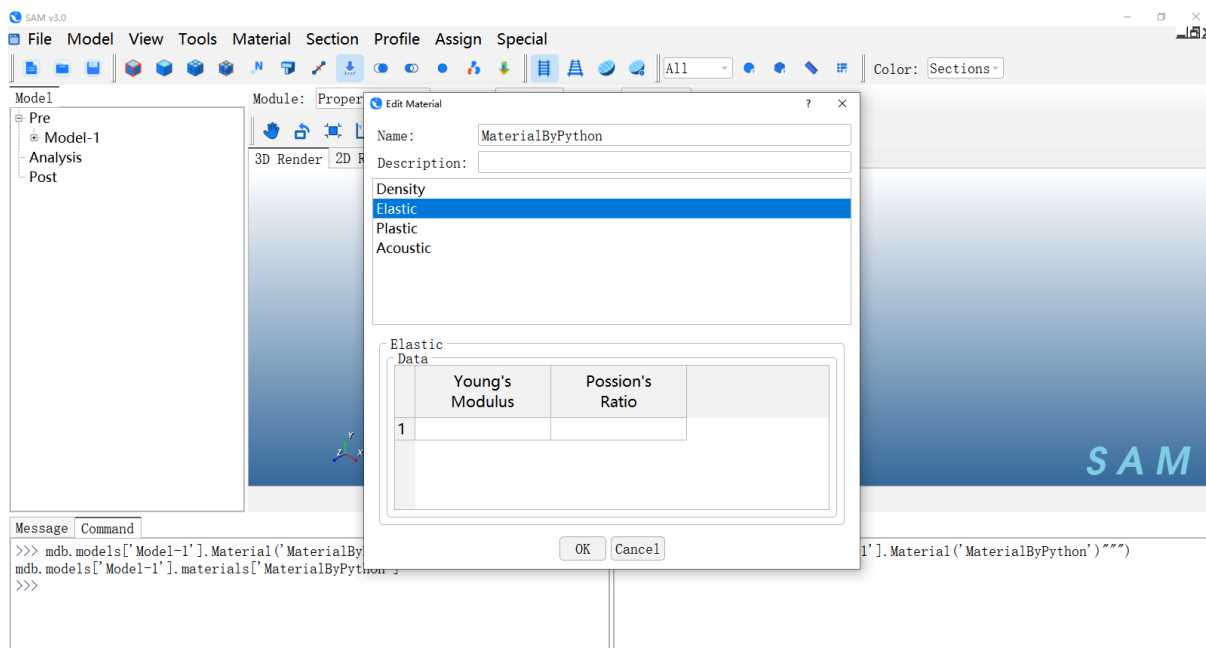


图 19 空数据材料

用户可尝试下面命令流，运行完毕继续通过上步 Manager 界面查看材料数据。

```
mdb.models['Model-1'].materials['MaterialByPython'].Density([[7800.0,]])
```

```
mdb.models['Model-1'].materials['MaterialByPython'].Elastic([[210000000000.0, 0.3]])
```

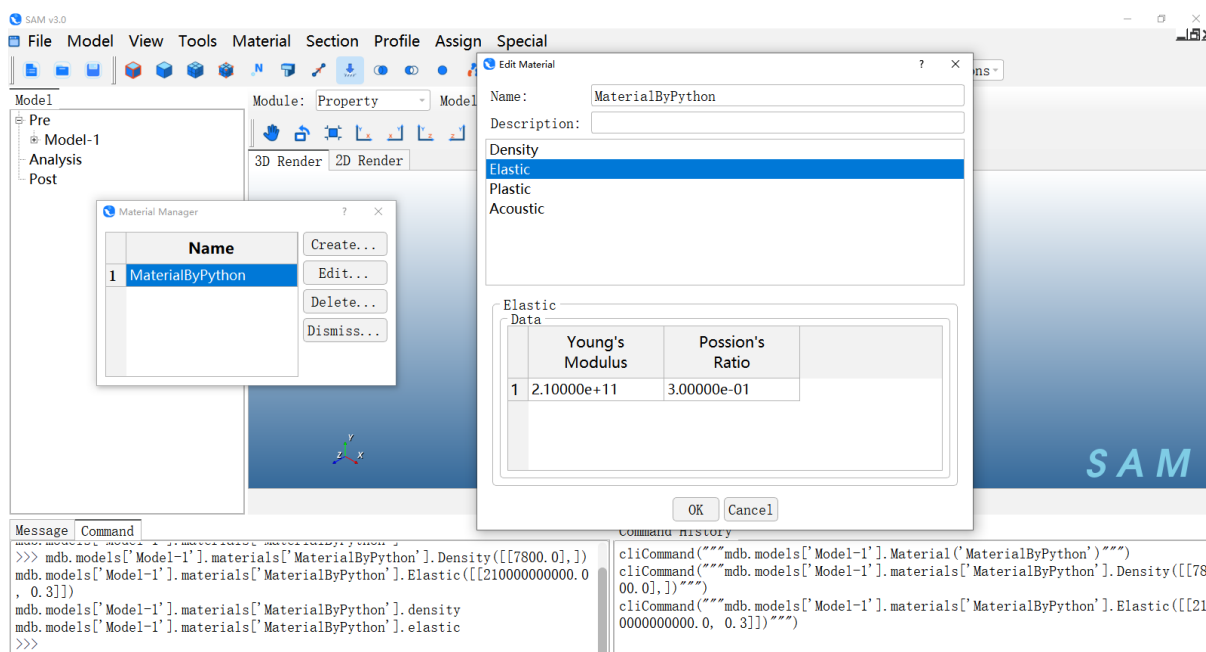


图 20 查看材料

2.2.3 实例 1：参数化创建加筋板前处理建模

2.2.3.1 实例说明

此实例为加筋板板架结构的前处理建模。

通过本实例用户将了解：

- SAM 软件中完成参数化的典型加筋板建模的脚本命令；
- SAM 中材料设置、分析步设置、边界条件和载荷加载脚本命令。

2.2.3.2 脚本命令及说明

#引用

```
from sam import *
```

```
from samConstants import *
```

#重置模型状态

```
Mdb()
```

#定义加筋板参数化建模所需参数

```
Dx=1
```

```
Nx=40
```

```
Dy=1
```

```
Ny=40
```

```
l=Dx*Nx
```

```
w=Dy*Ny
```

```
pressureMagnitude=1
```

#创建材料

```
mdb.models['Model-1'].Material(name='AISI 1020')
```

#材料密度

```
mdb.models['Model-1'].materials['AISI 1020'].Density(table=((7900.0,)), )
```

#材料杨氏模量和泊松比

```
mdb.models['Model-1'].materials['AISI 1020'].Elastic(table=((200000000000.0, 0.29)), )
```

#创建型材：T 型梁

```

mdb.models['Model-1'].TProfile(name='T_211-41', b=0.038, h=0.044, l=0.0, tf=0.005,
tw=0.005)
mdb.models['Model-1'].TProfile(name='XC214-4', b=0.111, h=0.069, l=0.0, tf=0.008,
tw=0.008)
#创建部件
mdb.models['Model-1'].Part(name='Part-1')
#部件变量
p = mdb.models['Model-1'].parts['Part-1']
#显示部件
vw.setDisplayedObject(p)
#构建加筋板参数化建模命令
GeomX=""
GeomY=""
#构建 X 方向上筋的个数及位置
for i in range(0,Nx+1):
    GeomX=GeomX+str(i*Dx)
    if i<Nx:
        GeomX=GeomX+", "
    continue

#构建 Y 方向上筋的个数及位置
for i in range(0,Ny+1):
    GeomY=GeomY+str(i*Dy)
    if i<Ny:
        GeomY=GeomY+", "
    continue

#参数化构建加筋板
p.BaseParametricStiffenedPlate(str(GeomX),str(GeomY),"0,0,0","0,0,0","0,0,0",0,1,0)

```

#基于型材属性创建梁截面

```
mdb.models['Model-1'].BeamSection(name='Section-beam-x',integration=DURING_
ANALYSIS, profile='T_211-41', material='AISI 1020')
```

```
mdb.models['Model-1'].BeamSection(name='Section-beam-y',integration=DURING_
ANALYSIS, profile='XC214-4', material='AISI 1020')
```

#创建壳 Section，定义板厚

```
mdb.models['Model-1'].HomogeneousShellSection(name='Section-shell',material='AI
SI 1020',thickness=0.1)
```

#获取指定 set 变量

```
regionX=mdb.models['Model-1'].parts['Part-1'].sets['Set-beam-x']
```

```
regionY=mdb.models['Model-1'].parts['Part-1'].sets['Set-beam-y']
```

#分配截面给指定 set 中包含的单元

```
p.SectionAssignment(region=regionX,sectionName='Section-beam-x',n1=(0,1,0))
```

```
p.SectionAssignment(region=regionY,sectionName='Section-beam-y',n1=(-1,0,0))
```

```
region=mdb.models['Model-1'].parts['Part-1'].sets['Set-shell']
```

```
mdb.models['Model-1'].parts['Part-1'].SectionAssignment(region=region,
sectionName='Section-shell')
```

#装配体变量

```
a = mdb.models['Model-1'].rootAssembly
```

#显示装配体

```
vw.setDisplayedObject(a)
```

```
p = mdb.models['Model-1'].parts['Part-1']
```

#基于部件创建实例

```
a.Instance(name='Part-1-1', part=p)
```

#创建静力分析步

```
mdb.models['Model-1'].StaticStep(name='Step-1',nlgeom=OFF,timeIncrementationM
ethod=AUTOMATIC,previous='Initial',timePeriod=1,initialInc=1,minInc=1E-005,maxIn
c=1,maxNumInc=100)
```

#获取实例中所有节点的集合变量

```
n1 = a.instances['Part-1-1'].nodes
```

```

#根据包围盒确定节点集合
nodes1 = n1.getByBoundingBox(-1.e-6, -1.e-6, -1.e-6, 1.e-6, w+1e-6, 1.e-6)
#创建节点 set
a.Set(nodes=nodes1, name='Set-1')
#获取 set 变量
set1 = mdb.models['Model-1'].rootAssembly.sets['Set-1']
#创建固支约束
mdb.models['Model-1'].EncastreBC(name='BC-1',createStepName='Step-1',region=
set1)
#装配体变量
a = mdb.models['Model-1'].rootAssembly
vw.setDisplayedObject(a)
#获取实例中所有单元的集合
f1 = a.instances['Part-1-1'].elements
#创建 surface 表面
a.Surface(side1Elements=f1,name='Surface-1')
#获取表面区域
region=a-surfaces['Surface-1']
#创建均布压力载荷
mdb.models['Model-1'].Pressure(name='Load-Pressure',createStepName='Step-1',re
gion=region,magnitude=pressureMagnitude)
#工作名称变量
jobName = 'Job-StiffenedPlate'
#创建工作任务
mdb.Job(name=jobName,model='Model-1',h5=ON)
#输出 inp 文件
mdb.jobs[jobName].writeInput()

```

2.2.4 实例 2：参数化加筋板计算及后处理

2.2.4.1 实例说明

此实例为加筋板板架结构的静力分析计算及后处理操作。

通过本实例用户将了解：

- SAM 中导入模型执行计算操作。
- SAM 中后处理操作。

2.2.4.2 脚本命令及说明

#导入模型

```
jobName = 'Job-StiffenedPlate'
```

```
inputFilePath = 'D:/Temp/Job-StiffenedPlate.inp'
```

```
mdb.ModelFromInputFile(name=jobName,inputFileName=inputFilePath)
```

#计算

```
mdb.jobs[jobName].submit()
```

#后处理结果名称变量

```
resultName = jobName + '.h5'
```

#加载后处理结果

```
session.importHDF5File(resultName)
```

#显示后处理结果

```
vw.setDisplayedObject(resultName)
```

#后处理数据变量

```
hdf5 = session.odbs[resultName]
```

#获取分析步名称

```
stepName = list(hdf5.steps.keys())[0]
```

#根据名称获取分析步变量

```
hdf5Step = hdf5.steps[stepName]
```

#获取增量步 Index

```
frameIndex = len(hdf5Step.frames) - 1
```

#获取增量步变量

```
frame = hdf5Step.frames[frameIndex]
```

```

#定义场量名称： 位移
fieldName_U = 'U'
#获取位移变量
field = frame.fieldOutputs[fieldName_U]
#定义分量名称： 幅值
componentName_U = 'Magnitude'
#刷新渲染窗口
vw.show()
#显示后处理结果
vw.setDisplayedObject(resultName)
#设置后处理显示的时间步与增量步
vw.odbDisplay.setFrame(str(stepName), frameIndex)
#设置后处理显示的场量与分量
vw.odbDisplay.setPrimaryVariable(str(fieldName_U), field.position, 1,
str(componentName_U))
#输出位移结果图片
vw.printPNGOffScreen("disp.png")
#定义场量名称： 应力
fieldName_S = 'S'
#获取应力变量
field1 = frame.fieldOutputs[fieldName_S]
#定义分量名称： 米塞斯力
componentName_S = 'Mises'
#设置后处理显示的场量与分量
vw.odbDisplay.setPrimaryVariable(str(fieldName_S), field1.position, 1,
str(componentName_S))
#输出应力结果图片
vw.printPNGOffScreen("mises.png")

```

2.3 界面二次开发

开发人员可基于标准模板开发 Python 界面，并通过动态添加的方式扩展到 SAM 软件中。

根据功能需要，软件功能界面以模块（Module）的方式注册到主界面中，每个模块中包含有对应的菜单栏和工具栏。

2.3.1 启动项

打开 SAM 软件安装目录下\Release，可通过右键点击桌面快捷方式图标，在右键菜单中点击“打开文件所在位置”。

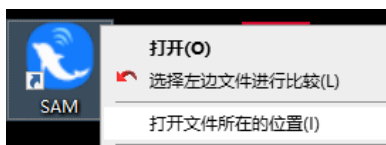


图 21 打开启动项目录

在启动项目录下找到批处理文件 samAPP.bat。

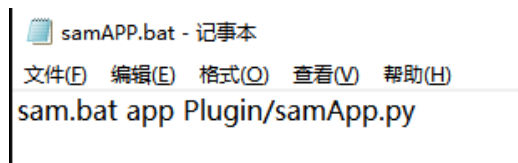


图 22 批处理脚本内容

此脚本内容含义为：

第一个参数 **sam.bat**，执行启动目录下 **sam.bat** 批处理脚本；

第二个参数 **app**，使用 **app** 模式启动 SAM 软件，此模式下会根据第三个参数加载初始化脚本；

第三个参数指定初始化脚本为启动项目录下 **Plugin/samApp.py**。

2.3.2 通过脚本初始化需要加载的模块

界面二次开发初始化脚本为启动项目录下 **Plugin/samApp.py**。

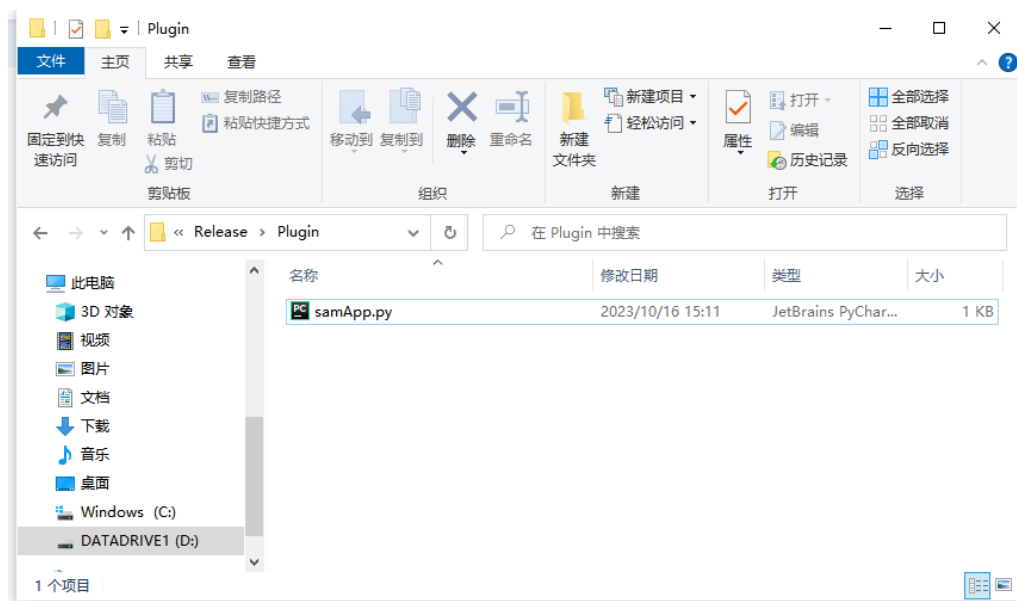


图 23 初始化脚本

脚本内容如下：

#引用 python os 库和 sys 库

#sys 模块负责处理 Python 运行时的配置及资源，从而可以与系统环境交互

#os 模块负责程序与操作系统的交互，提供了访问操作系统底层的接口

```
import os, sys
```

```
sys.path.append(os.path.abspath('__file__'))
```

#引用 SAM 的界面库

```
from FEGuiPy import *
```

#初始化 SAM 程序

```
app = FEApp()
```

```
app.init(sys.argv)
```

#初始化 SAM 主界面

```
mw = FEMainWindow(app)
```

#注册 SAM 界面模块，默认为 SAM 自带的基础模块

```
mw.registerModule("Part", "Part")
```

```
mw.registerModule("Property", "PropertyModuleGui")
```

```
mw.registerModule("Assembly", "AssemblyModuleGui")
```

```
mw.registerModule("Step", "StepModuleGui")
mw.registerModule("Interaction", "InteractionModuleGui")
mw.registerModule("Load", "LoadModuleGui")
mw.registerModule("Mesh", "MeshModuleGui")
mw.registerModule("Job", "JobModuleGui")
mw.registerModule("Visualization", "VisualizationModuleGui")
#创建 SAM 程序
app.create()
#执行 SAM 程序
app.run()
```

通过上述脚本内容可看出，**SAM** 可通过在初始化脚本中注册模块的方式将开发的界面添加到程序主界面中，其注册语句 **registerModule** 需要两个参数，第一个参数为注册的模块名称，第二个参数为注册的模块类。

开发人员可以在注册模块语句中加入自己想要注册的模块信息，语句的顺序决定了软件打开后界面上模块的排布顺序。

我们在 **Visualization** 模块后添加一个测试示例模块：

```
mw.registerModule("CustomModule", "CustomModuleGui")
```

添加完语句后，软件在启动时会在初始化脚本所在目录（启动项目录/**Plugin**）下搜索模块内容，因此我们要在此目录下编写对应模块内容，否则软件无法启动。具体模块内容格式请参看下一章节。

2.3.3 模块

SAM 的界面模块包含界面定义与语句。

在软件安装目录下 **Example/SecondaryDevelopment** 文件夹中附有一个界面模块示例脚本 **CustomModuleGui.py**，我们将此脚本拷贝到软件启动项目录下 **Plugin** 目录。

```

from FEGuiPy import *
from PySide2.QtWidgets import QMenu, QMessageBox
from PySide2.QtCore import QObject, Signal, Slot

class CustomToolsetGui(FEToolsetGui):
    def __init__(self):
        FEToolsetGui.__init__(self, "Custom")
        menu = QMenu("Custom", self)
        menu.addAction("SayHello...", self.SayHello)

    @Slot()
    def SayHello(self):
        box = QMessageBox()
        box.warning(self, "Custom", "Hello World!")

customToolset = CustomToolsetGui()

class CustomModuleGui(FEModuleGui):
    def __init__(self):
        FEModuleGui.__init__(self, "CustomModule", FEModuleGui.PART)
        self.registerToolset(customToolset, GUI_IN_MENUBAR)

customModule = CustomModuleGui()

```

图 24 界面模块示例

下文我们将介绍如何编写界面模块内容。

2.3.3.1 引用

界面模块需要引用的库一般分为 2 类：

- SAM 提供的动态库，如界面库 FEGuiPy 和数据访问库 kernelAccess；
- Qt 提供的 PySide 库，比较常用的有 PySide2.QtWidgets、PySide2.QtCore、PySide2.QtGui 等。

我们可以根据界面需求来决定引用哪些库。本示例中我们需要在界面上实现菜单栏及按钮，点击对应按钮会执行相应的命令，因此我们需要引用的库如下：

```

from FEGuiPy import *
from PySide2.QtWidgets import QMenu, QMessageBox
from PySide2.QtCore import QObject, Signal, Slot
from samConstants import *

```

2.3.3.2 模块类

界面模块的添加需要定义一个模块类，在 SAM 中界面模块类继承自模板类 FEModuleGui，名称与启动项目目录下 Plugin/samApp.py 中注册的模块类名称（第二个参数）相同。其定义如下：

```
class CustomModuleGui(FEModuleGui):
```

```
    def __init__(self):
```

```
        FEModuleGui.__init__(self, "CustomModule", FEModuleGui.PART)
```

```
customModule = CustomModuleGui()
```

在类初始化函数中，调用父类的初始化函数，传入 3 个参数：

- **self**，模块类对象；
- 注册的模块名称，与启动项目目录下 `Plugin/samApp.py` 中注册的模块名称（第一个参数）相同；
- 显示对象，规定此模块下模型显示类型，格式为 `FEModuleGui.Type`，其中 `Type` 为枚举值，包含 **PART**（部件显示）、**ASSEMBLY**（装配体显示）、**ODB**（后处理显示）3 种类型。

完成类定义后，我们保存文件，双击 **SAM** 启动项目目录下的批处理脚本 `samAPP.bat`，可查看模块类添加的效果，如图所示，模块下拉框中已经添加了我们定义的模块。

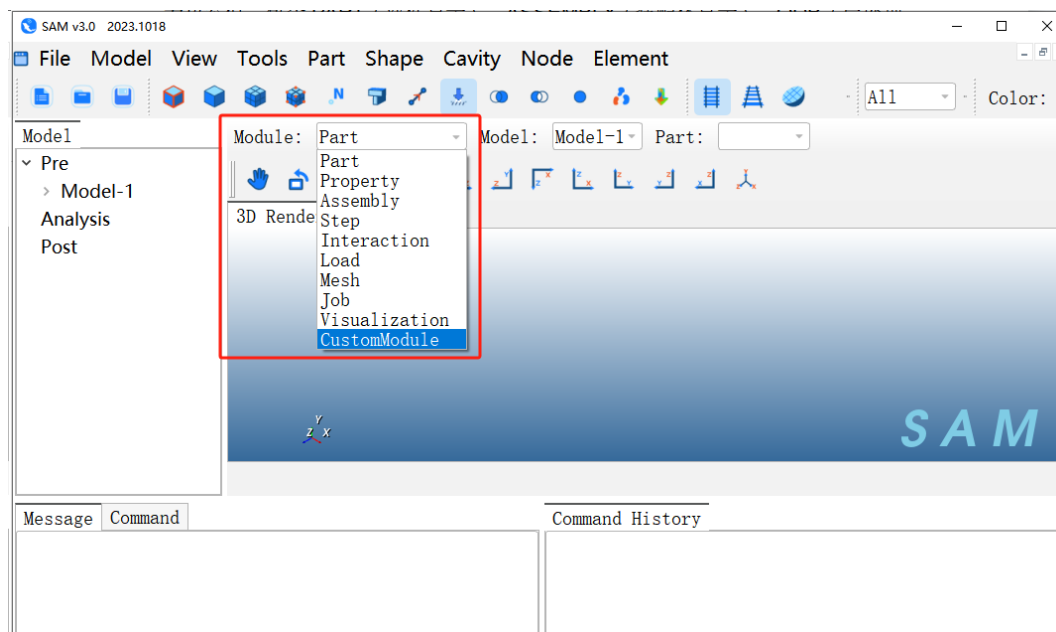


图 25 添加示例模块

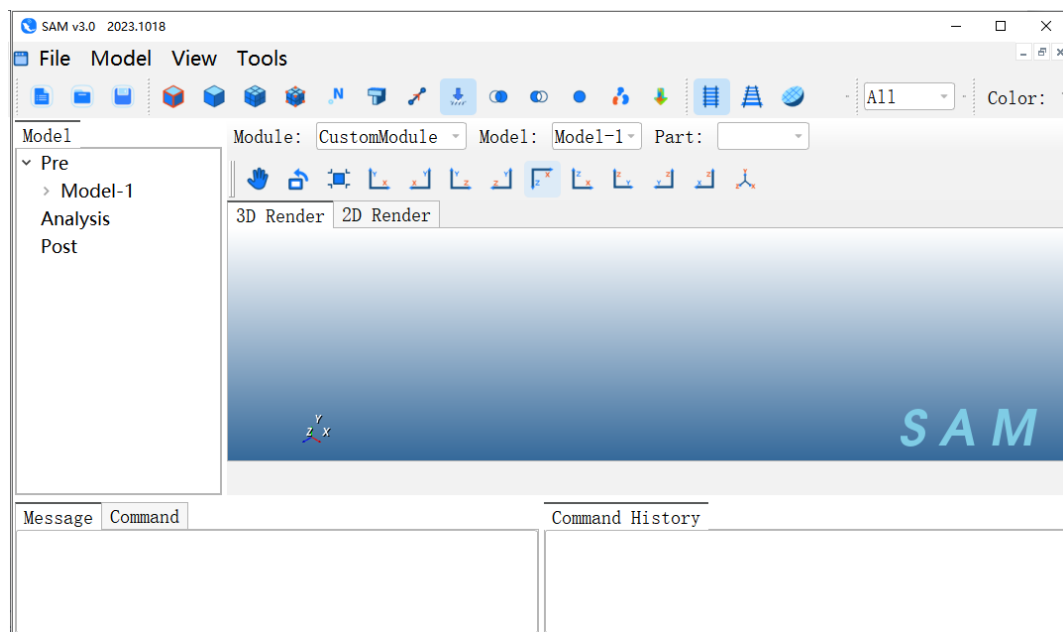


图 26 切换到示例模块

下面我们将介绍如何给示例模块中添加自定义菜单栏、工具栏等内容。

2.3.3.3 菜单栏及工具栏

要实现自定义菜单栏、工具栏的创建，需要定义一个菜单栏类，在 SAM 中继承自模板类 FEToolsetGui。

```
class CustomToolsetGui(FEToolsetGui):
```

```
    def __init__(self):
```

```
        FEToolsetGui.__init__(self, "Custom")
```

#定义类对象

```
customToolset = CustomToolsetGui()
```

在类初始化函数中，调用父类的初始化函数，传入 2 个参数：

- self，菜单栏类对象；
- 类名称。

完成初始化后，具体的界面设计与槽函数定义写在我们定义的菜单栏中：

```
class CustomToolsetGui(FEToolsetGui):
```

```
    def __init__(self):
```

```
        FEToolsetGui.__init__(self, "Custom")
```

```
        menu = QMenu("Custom", self)
```

```
menu.addAction("SayHello...", self.SayHello)
```

```
@Slot()
```

```
def SayHello(self):
```

```
    box = QMessageBox()
```

```
    box.warning(self, "Custom", "Hello World!")
```

✧ 由于语法规则，菜单栏类定义和对象定义需要写在模块类定义之前。

要在模块中添加我们定义的菜单栏及工具栏，需要在模块类中添加一行代码：

```
self.registerToolset(customToolset, GUI_IN_MENUBAR)
```

接口函数需要传入 2 个参数：

- 类对象；
- 界面类型，用枚举值表示需要向界面中添加何种类型的界面元素，常用的有 **GUI_IN_MENUBAR**（菜单栏）和 **GUI_IN_TOOLBAR**（工具栏）。

完成类定义后，我们保存文件，双击 **SAM** 启动项目目录下的批处理脚本 **samAPP.bat**，可查看菜单栏添加的效果，如图所示：

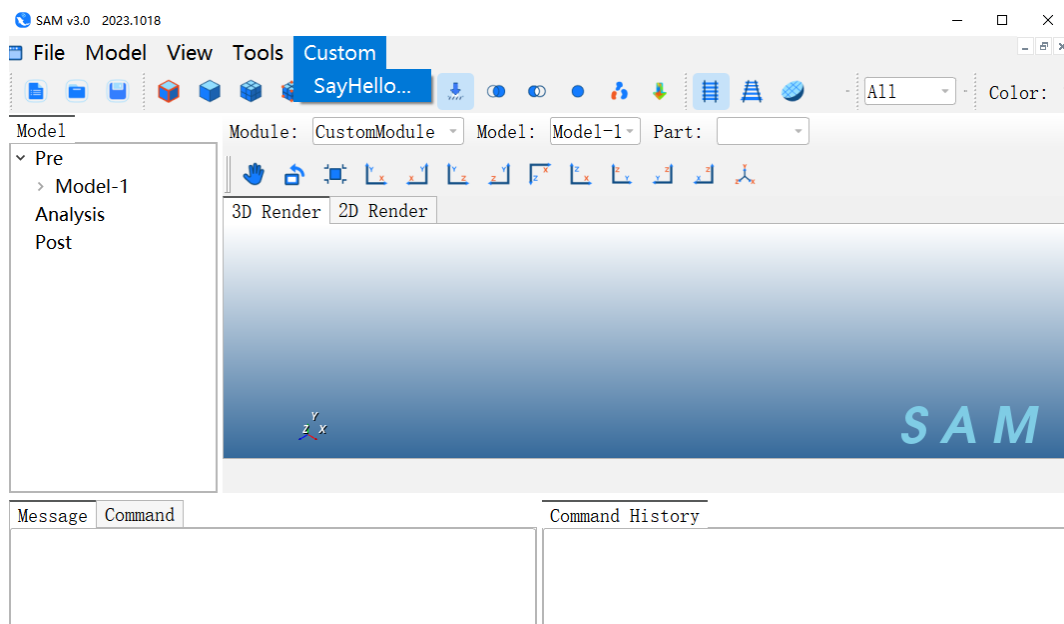


图 27 模块下自定义菜单栏

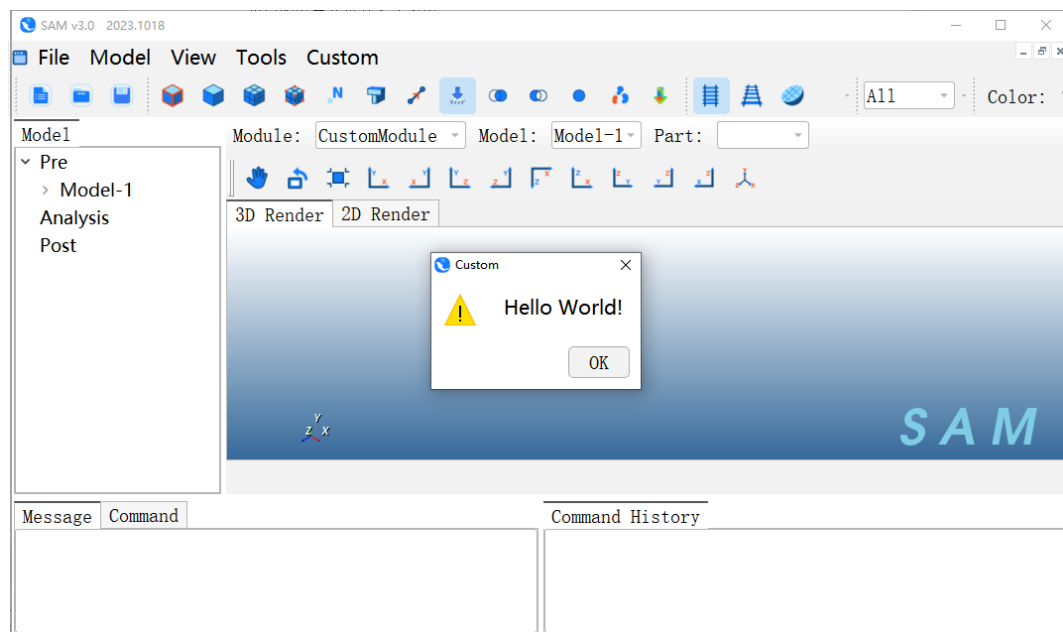


图 28 点击自定义按钮

2.3.3.4 发送内核命令

我们可以在界面模块中使用 SAM 的 Python 接口向 SAM 内核发送命令，以实现对外核数据的操作。

我们在自定义的菜单栏中新增一个按钮，向内核发送一个参数化建模的命令（高亮部分代码）。

```
class CustomToolsetGui(FEToolsetGui):
    def __init__(self):
        FEToolsetGui.__init__(self, "Custom")
        menu = QMenu("Custom", self)
        menu.addAction("SayHello...", self.SayHello)
        menu.addAction("Create Cylinder", self.CreateCylinder)

    @Slot()
    def SayHello(self):
        box = QMessageBox()
        box.warning(self, "Custom", "Hello World!")
```

@Slot()

def CreateCylinder(self):

from kernelAccess import mdb

p=mdb.models['Model-1'].parts['Part-1']

p.BaseParametricCylinder(5,10,360,5,15,OFF)

保存文件后，双击 SAM 启动项目目录下的批处理脚本 samAPP.bat，打开软件。

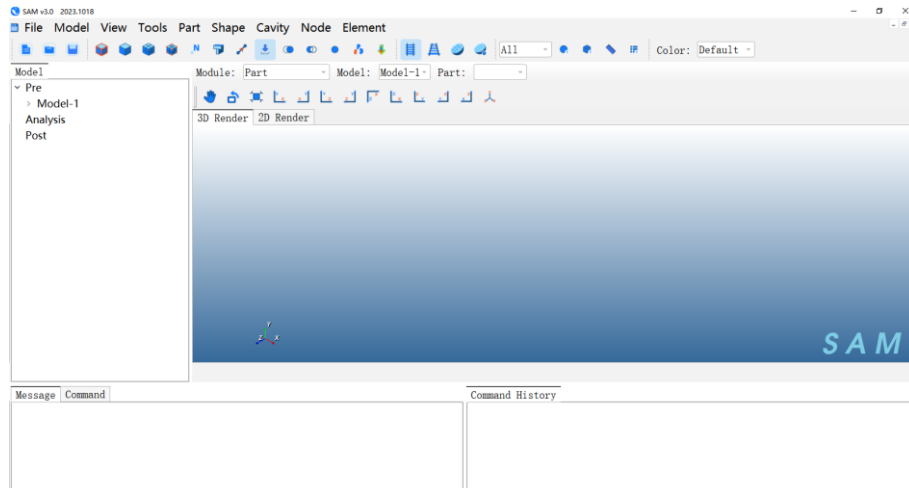


图 29 打开软件

点击 Part->Create，打开 Part 创建窗口，点击 OK，完成 Part 的创建。

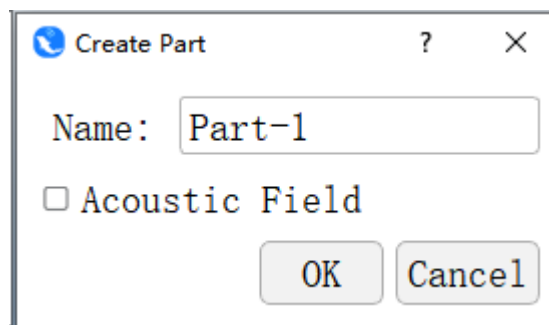


图 30 创建 Part

然后在 Module 下拉框中，切换到我们创建的 CustomModule 模块。

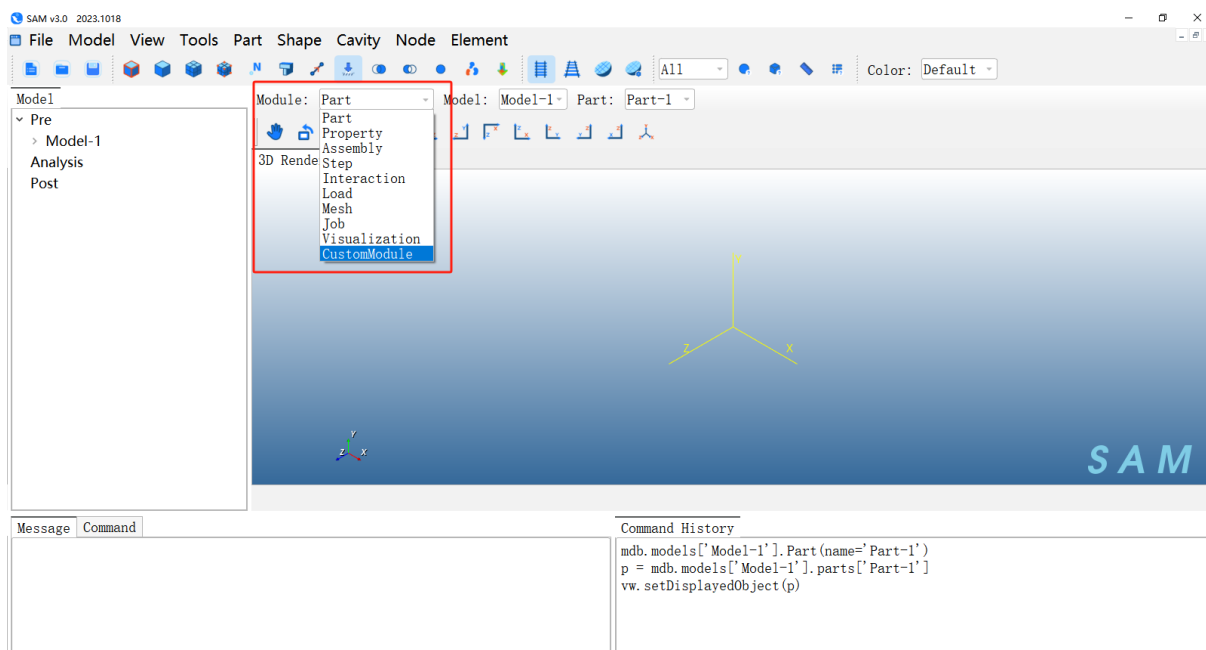


图 31 创建 Part

点击 Custom->Create Cylinder，即可发送我们设置好的命令，可以看到界面上创建了一个开口圆柱壳网格模型。

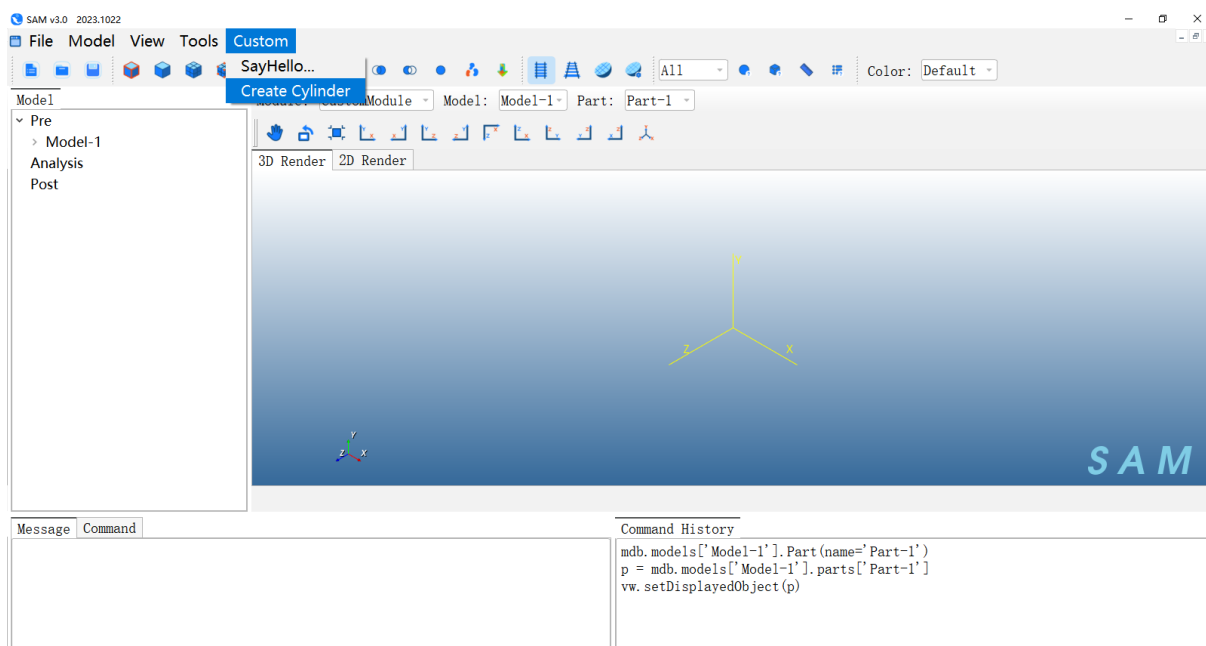


图 32 点击按钮

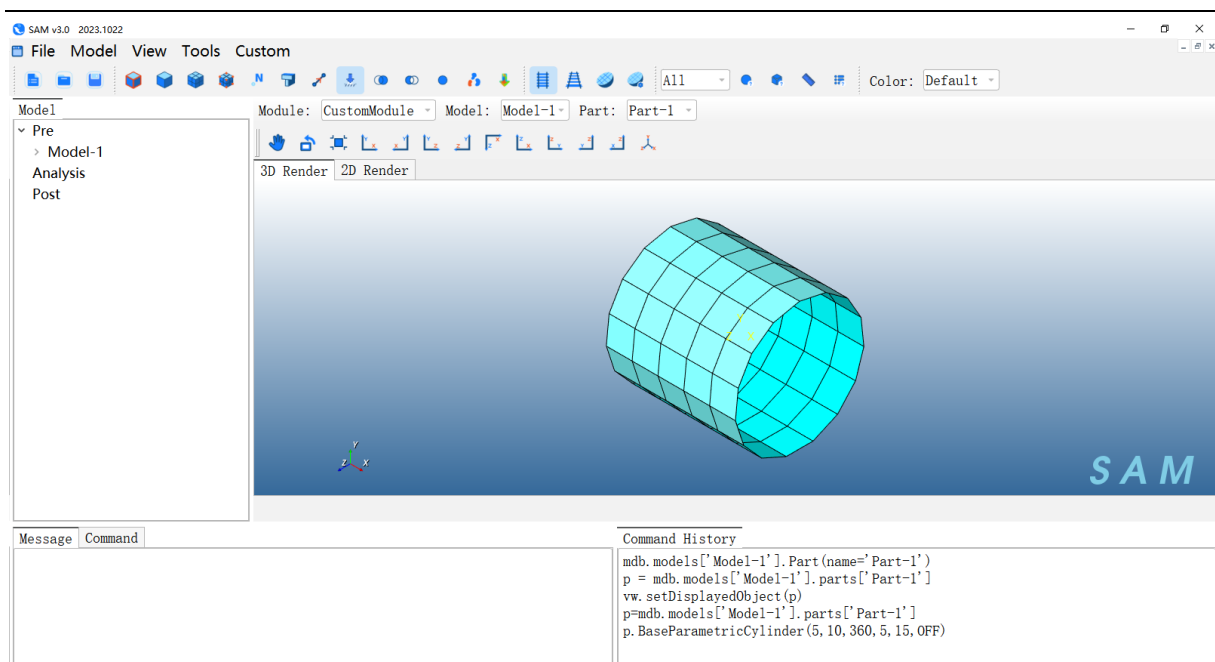


图 33 发送命令，创建网格模型

3 C++二次开发

接下来介绍如何使用 SAM 提供的接口编写动态库以及将动态库与界面进行关联。

3.1 工程构建及环境配置

SAM 提供了一套开发环境供开发人员访问核心数据及调用各种接口，使用 **cmake** 工具管理工程。开发人员可根据 **cmakelist** 快速构建工程和配置开发环境。

开发环境配置如下：

工具/环境	版本
Visual C++ Compiler	2015
Qt	5.12.6
Hdf5	1.12
VTK	8.2
Cmake	3.15

为了保持工程目录的一致性，SAM 二次开发库代码一般遵循下文介绍的步骤进行构建。

3.1.1 构建主目录

在系统盘以外的硬盘（如 D 盘）构建工程主目录，这里我们在 D 盘下新建一个文件夹 SAMDevelopment（开发人员可根据项目需要自定义命名）。

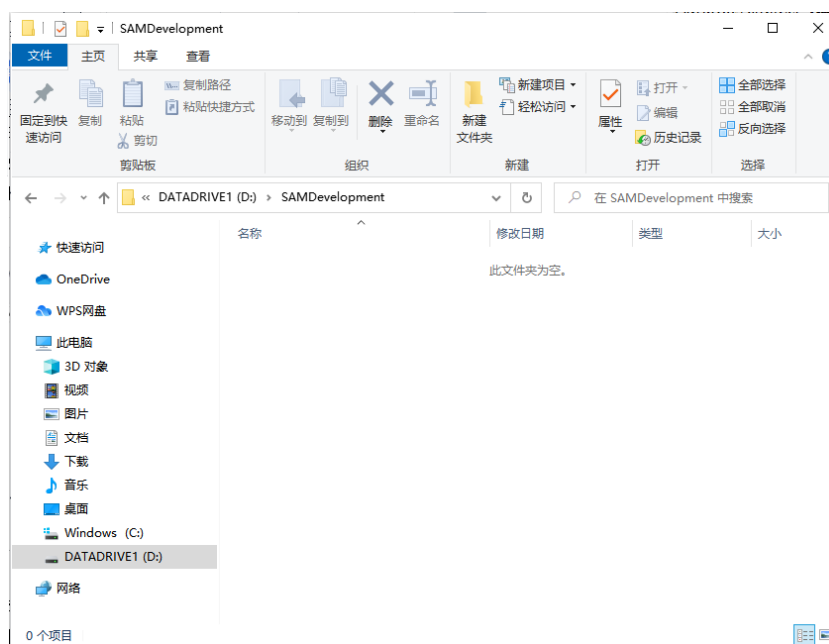


图 34 新建文件夹

3.1.2 配置 SAM 开发环境

将 SAM 开发环境（Platform）放在此文件夹下，其中 bin 目录为 SAM 启动目录，包括 Debug 与 Release 版本的动态库；include 目录包含 SAM 接口头文件；libs 目录包含 Debug 与 Release 版本的 lib 静态库；ThridPartys 目录包含 SAM 使用到的第三方库。

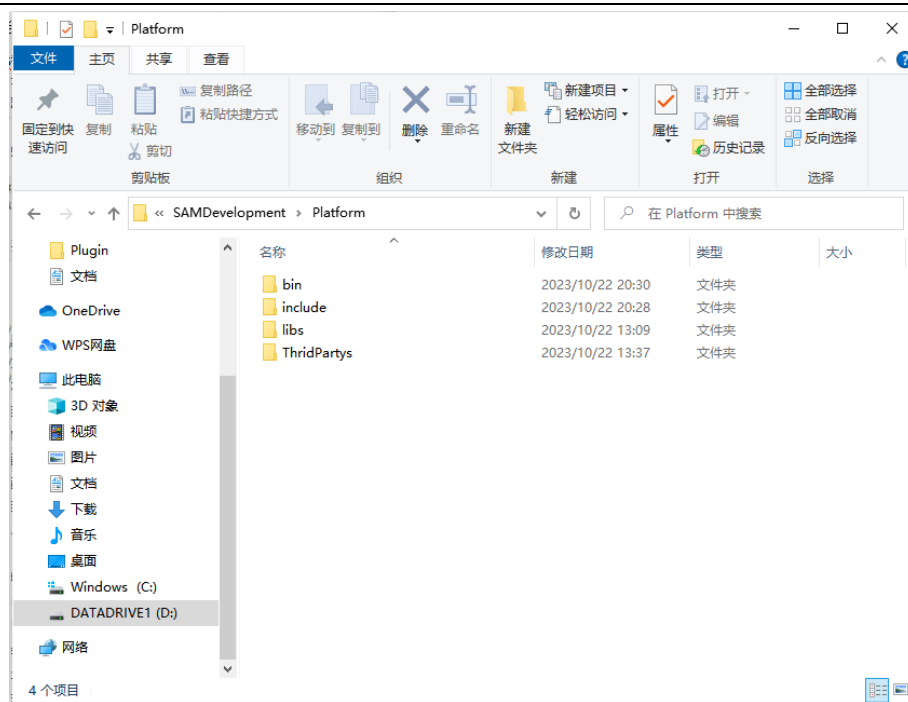
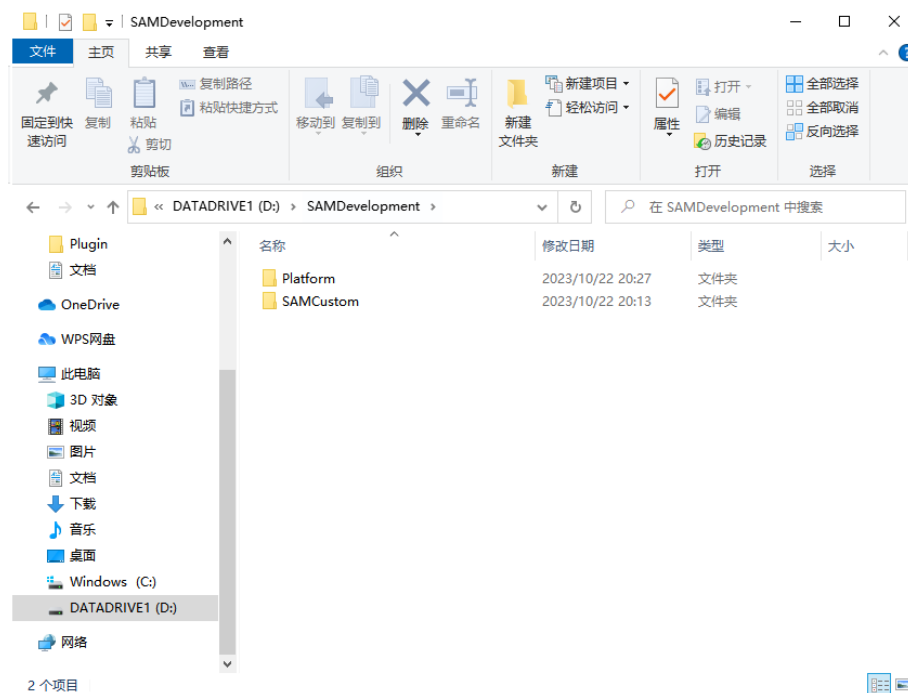


图 35 SAM 开发环境

3.1.3 构建工程目录

在 Platform 文件夹同级目录下新建一个工程目录 SAMCustom(根据项目需要自行修改名字)，在此文件夹下新建 src 文件夹用来管理源代码；新建 bin 文件夹作为工程的输出文件目录。



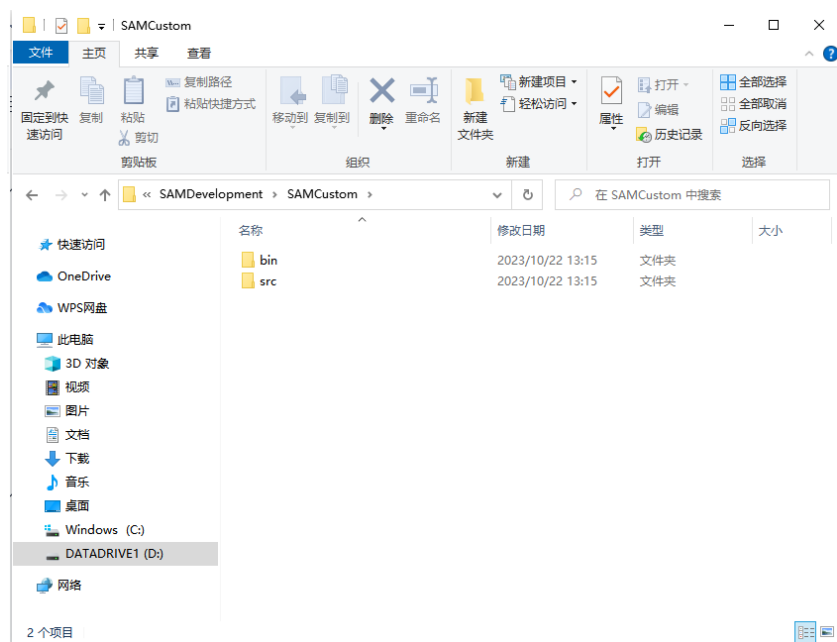


图 36 新建代码目录和输出目录

3.1.4 管理工程目录

在工程目录下新建一个 CMakeLists.txt 以管理项目。

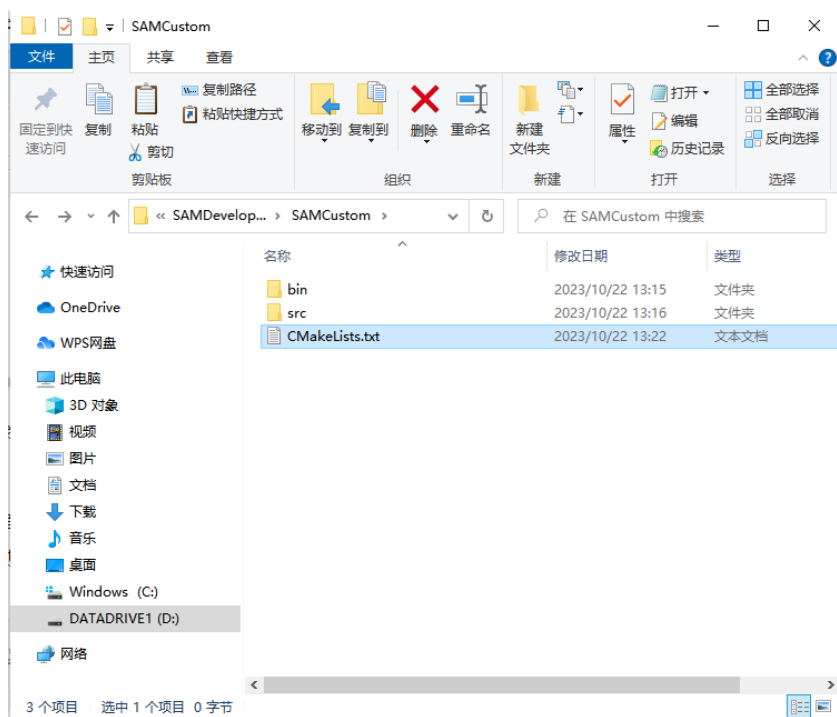


图 37 创建 CMakeLists

其内容如下：

```
cmake_minimum_required(VERSION 3.0.0)
```

```
#源代码生成中禁用
set(CMAKE_DISABLE_IN_SOURCE_BUILD ON)
#生成 lib
set(CMAKE_WINDOWS_EXPORT_ALL_SYMBOLS ON)

project(Custom)
#根据当前 cmake 目录找到最外层开发目录
get_filename_component(current_dir ${CMAKE_CURRENT_SOURCE_DIR} DIRECTORY)

# set libs
set(LIBS_FEASOFT_ROOT ${current_dir}/Platform)
set(LIBS_PYTHON_ROOT ${current_dir}/Platform/bin/python27)
set(LIBS_QT5_ROOT ${LIBS_FEASOFT_ROOT}/ThridPartys/Qt-5.12.6-VC14-64)
set(LIBS_TCMALLOC_ROOT ${LIBS_FEASOFT_ROOT}/ThridPartys/TCMalloc)

set(CMAKE_PREFIX_PATH ${CMAKE_PREFIX_PATH} ${LIBS_QT5_ROOT}/lib/cmake/Qt5)
set(CMAKE_PREFIX_PATH ${CMAKE_PREFIX_PATH} ${LIBS_QT5_ROOT}/lib/cmake/Qt5Widgets)
set(CMAKE_PREFIX_PATH ${CMAKE_PREFIX_PATH} ${LIBS_QT5_ROOT}/lib/cmake/Qt5Core)
set(CMAKE_PREFIX_PATH ${CMAKE_PREFIX_PATH} ${LIBS_QT5_ROOT}/lib/cmake/Qt5Gui)

# check Qt library
find_package(Qt5Widgets REQUIRED)
find_package(Qt5Core REQUIRED)
find_package(Qt5Gui REQUIRED)

# output paths
set(CMAKE_INSTALL_PREFIX ${CMAKE_CURRENT_SOURCE_DIR})
set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_INSTALL_PREFIX}/bin)
set(CMAKE_LIBRARY_OUTPUT_DIRECTORY ${CMAKE_INSTALL_PREFIX}/lib)
set(CMAKE_ARCHIVE_OUTPUT_DIRECTORY ${CMAKE_INSTALL_PREFIX}/lib)

#project
add_subdirectory(src/Custom)
```

3.1.5 新建项目及项目管理

在 `src` 下新建项目，每个 `src` 下的文件夹都是一个代码项目。此处我们新建一个 `Custom` 文件夹，在目录中创建一个 `CMakeLists.txt` 以管理项目代码。

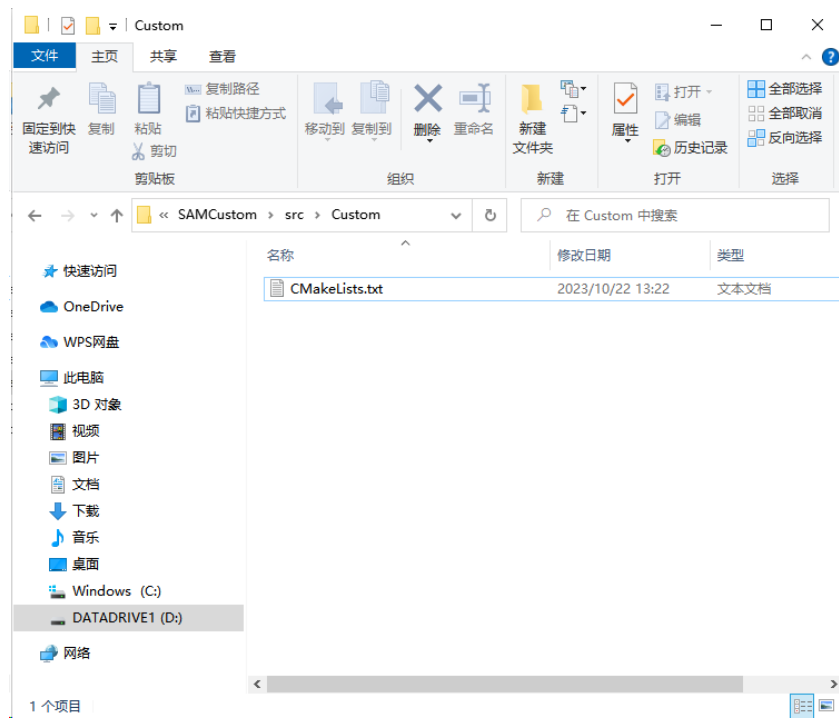


图 38 创建 CMakeLists

其内容如下，高亮标出的是 SAM 环境：

```
# directories
include_directories(
    ${LIBS_QT5_ROOT}/include
    ${LIBS_QT5_ROOT}/include/QtCore
    ${LIBS_PYTHON_ROOT}/include
    ${LIBS_FEASOFT_ROOT}/include

    ${CMAKE_INSTALL_PREFIX}/src/Custom
)

link_directories(
    ${LIBS_QT5_ROOT}/lib
    ${LIBS_FEASOFT_ROOT}/libs
    ${LIBS_TCMALLOC_ROOT}
    ${LIBS_PYTHON_ROOT}/libs
    ${CMAKE_ARCHIVE_OUTPUT_DIRECTORY}
)

# source list
file(GLOB TARGET_SRC
    "${CMAKE_CURRENT_SOURCE_DIR}/*.cpp"
    "${CMAKE_CURRENT_SOURCE_DIR}/*.h"
```

```
)

# dynamic library
add_library(Custom SHARED
    ${TARGET_SRC}
)

# compiler settings
set_property(TARGET Custom
    PROPERTY      COMPILE_DEFINITIONS      _CRT_SECURE_NO_WARNINGS
    _CRT_NON_CONFORMING_SWPRINTFS
)

set_target_properties(Custom PROPERTIES LINK_FLAGS /INCLUDE:"__tcmalloc")

add_definitions(
    -DUNICODE
    -D_UNICODE
)

# linker settings
set_target_properties(Custom PROPERTIES
    SUFFIX ".pyd")

target_link_libraries(Custom
    debug Qt5Core.lib
    debug SAMBasicUtilities
    debug SAMBasicCoreUtilities
    debug SAMBasicContainers
    debug SAMBasicPrimitives
    debug SAMPrimaryObjects
    debug SAMCoreMeshDefs
    debug SAMMeshInterfaceObjects
    debug SAMKernelAttributesInterfaceObjects
    debug SAMQtRemoteGlobalSource
    debug SAMRenderInterface
    debug SAMModelDatabase
    debug FEMathUtils
    debug SAMGeometryCoreModule
    debug SAMKernelAttributesInterfaceObjects
    debug SAMGeometryComponent
    debug SAMKernel
    debug SAMPythonInterfaces
    debug SAMPythonModuleConfiguration
    debug SAMPythonUtilities
```



```

        debug libtcmalloc_minimal.lib
    )

    target_link_libraries(Custom
        optimized Qt5Core.lib
        optimized SAMBasicUtilities
        optimized SAMBasicCoreUtilities
        optimized SAMBasicContainers
        optimized SAMBasicPrimitives
        optimized SAMPrimaryObjects
        optimized SAMCoreMeshDefs
        optimized SAMMeshInterfaceObjects
        optimized SAMKernelAttributesInterfaceObjects
        optimized SAMQtRemoteGlobalSource
        optimized SAMRenderInterface
        optimized SAMModelDatabase
        optimized FEMathUtils
        optimized SAMGeometryCoreModule
        optimized SAMKernelAttributesInterfaceObjects
        optimized SAMGeometryComponent
        optimized SAMKernel
        optimized SAMPythonInterfaces
        optimized SAMPythonModuleConfiguration
        optimized SAMPythonUtilities
        optimized libtcmalloc_minimal.lib
    )

```

3.1.6 添加代码文件

开发人员在 `src` 下项目目录中新建代码源文件及头文件。为了让我们编写的动态库能够注册到 SAM 内核中，我们需要实现 3 个基于 SAM 接口的代码文件，具体内容如下：

3.1.6.1 CustomPytModule

继承自接口类 `pyoModule`，添加模块数据到模块接口

头文件 `CustomPytModule.h`：

```

/* -*- mode: c++ -*- */
#ifndef customPytModule_h
#define customPytModule_h

// Forward declarations

```

```
class omuArguments;
```

```
// Includes
```

```
// Begin local includes
```

```
#include <pyoModule.h>
```

```
// Class definition
```

```
class customPytModule : public pyoModule
```

```
{
```

```
public:
```

```
    customPytModule();
```

```
    virtual ~customPytModule();
```

```
    virtual void DefineConstants();
```

```
private:
```

```
    customPytModule(const customPytModule&);
```

```
    customPytModule& operator=(const customPytModule&);
```

```
};
```

```
#endif // #ifdef customPytModule_h
```

源文件 CustomPytModule.cpp:

```
#include <customPytModule.h>
```

```
#include <omuArguments.h>
```

```
static omuInterfaceObj::methodTable customPytModuleMethods[] =
```

```
{
```

```
    {0, 0}
```

```
};
```

```
customPytModule::customPytModule()
```

```
    : pyoModule("Custom", customPytModuleMethods, pyoModule::NO_IMPORT)
```

```
{
```

```
    //第一个参数与项目输出的库名称相同
```

```
}
```

```
customPytModule::~~customPytModule()
```

```
{
```

```
    // todo
```

```
}
```

```
void customPytModule::DefineConstants()
{
    // todo
}
```

3.1.6.2 CustomUtils

模块工具类，用来注册我们编写的段落，包括初始化与回收。

头文件 CustomUtils.h:

```
#ifndef customUtils_h
#define customUtils_h

void SAMCustom_initialize();
#endif
```

源文件 CustomUtils.cpp:

```
#include <customUtils.h>

#include <Python.h>
#include <customPytModule.h>
#include <iniPythonModuleRegistrar.h>
#include <SAMCustomFragment.h>

static int samCustomSpecCount = 0;
static customPytModule* customPyd = 0;
static SAMCustomFragment* SAMCustom = 0;

void SAMCustom_specific_initialize(int& count)
{
    if (!samCustomSpecCount++)
    {
        ++count;
        customPyd = new customPytModule();
        SAMCustom = new SAMCustomFragment();
    }
}

void SAMCustom_specific_finalize(int& count)
{
    if (!--samCustomSpecCount)
    {
        --count;
        if (SAMCustom)
        {
```

```

        delete SAMCustom;
        SAMCustom = 0;
    }
    if (customPyd)
    {
        delete customPyd;
        customPyd = 0;
    }
}

static int samCustomCount = 0;

void SAMCustom_initialize(int& count)
{
    if (!samCustomCount++)
    {
        ++count;
        SAMCustom_specific_initialize(count);
    }
}

void SAMCustom_finalize(int& count)
{
    if (--samCustomCount)
    {
        --count;
        SAMCustom_specific_finalize(count);
    }
}

PyMODINIT_FUNC initCustom(void)
{
    iniPythonModuleRegistrar::Instance().Register(SAMCustom_initialize, SAMCustom_finalize);
}

```

3.1.6.3 SAMCustomFragment

根据代码需要，继承自 SAM 提供的核心数据接口，此处我们对 part 数据进行操作，因此代码继承自 ptsKPartFragment。此文件初始化了一个命令，执行该命令可在模型的部件中绘制出一个由 S4R 单元构成的矩形网格模型。

头文件 SAMCustomFragment.h:

```

/* -*- mode: c++ -*- */
#ifndef SAMCustomFragment_h
#define SAMCustomFragment_h

```

```

#include <CustomUtils.h>

class omuArguments;

// Class definition

class SAMCustomFragment : public ptsKPartFragment
{
public:
    SAMCustomFragment();
    virtual ~SAMCustomFragment();

    omuPrimitive* Copy() const;

    omuPrimitive* CreateRectangle(omuArguments&) const;
};

#endif // #ifndef SAMCustomFragment_h

```

源文件 SAMCustomFragment.cpp:

```

#include <SAMCustomFragment.h>
#include <ptoKPart.h>
#include <bmeMesh.h>
#include <omeMesh.h>
#include <mesUtils.h>
#include <bmeElementClass.h>
#include <shpShape.h>
#include <bmgUtils.h>
#include <samMdbDrawable.h>
#include <samModelDrawable.h>
#include <samPartDrawable.h>

static omuInterfaceObj::methodTable SAMCustomFMethods[] =
{
    {"CreateRectangle",
    (omuInterfaceObj::methodFunc)&SAMCustomFragment::CreateRectangle},
    { 0, 0 }
};

static omuInterfaceObj::memberTable SAMCustomFMembers[] =
{
    { 0, 0, 0 }
};

```

```
SAMCustomFragment::SAMCustomFragment()
: ptsKPartFragment()
{
    omuInterfaceObj::DescribeType("SAMCustomFragment", SAMCustomFMethods,
SAMCustomFMembers);
}
SAMCustomFragment::~~SAMCustomFragment()
{
    // todo
}

omuPrimitive* SAMCustomFragment::Copy() const
{
    return new SAMCustomFragment(*this);
}

omuPrimitive* SAMCustomFragment::CreateRectangle(omuArguments& args)
{
    double width = 5.0;
    double length = 10.0;
    int widthScale = 5;
    int lengthScale = 10;

    args.Begin();
    args.Optional();
    args.Get(width, "width");
    args.Get(length, "length");
    args.Get(widthScale, "widthScale");
    args.Get(lengthScale, "lengthScale");
    args.End();

    ftrFeatureList* fl = GetPart()->GetFeatureList();
    double fPI = 3.141592657;
    // 构建网格数据
    int i;
    int j;
    // 节点数据
    float currentXRadian;
    float currentYRadian;
    float x = 0;
    float y = 0;
    float z = 0;
    currentYRadian = width / widthScale;
    currentXRadian = length / lengthScale;
```

```

int totalNodeCount = (widthScale + 1) * (lengthScale + 1);
QVector<int> nodeLabels(totalNodeCount);
QVector<float> coords(totalNodeCount * 3);
int* nodeLabelArr = nodeLabels.data();
float* coordArr = coords.data();
int nodeStartIndex = 0;
int nodeLabel = 1;
utiCoordCont3D coordContainer = utiCoordCont3D();
int elemLabel = 1;
int numClasses = 0;
bmeElementClass** classes;
const bmeMesh* objectMesh = fl->ConstGetMesh(bdoDefaultInstId);
if (objectMesh) {
    const bmeNodeData& nodeData = objectMesh->NodeData();
    const bmeElementData& elementData = objectMesh->ElementData();
    nodeStartIndex = nodeData.NumNodes();
    coordContainer.Append(nodeData.CoordContainer());

    cowListInt userNodeLabels;
    nodeData.GetUserNodeLabels(userNodeLabels);
    nodeLabels.resize(nodeStartIndex + totalNodeCount);
    nodeLabelArr = nodeLabels.data();
    for (int i = 0; i < nodeStartIndex; i++) {
        *nodeLabelArr++ = userNodeLabels.Get(i);
    }
    nodeLabel = nodeData.NextLabel();
    elemLabel = elementData.NumElements() + 1; // .GetNextAvailableLabel();
    numClasses = elementData.NumClasses();
    classes = (bmeElementClass**)malloc(sizeof(bmeElementClass*) * (numClasses + 1));
    for (int i = 0; i < numClasses; i++) {
        classes[i] = bmeElementClass::ConstructObject(elementData.GetClass(i));
    }
}
else {
    classes = (bmeElementClass**)malloc(sizeof(bmeElementClass*) * 1);
}
int nodeId = 0;
for (i = 0; i < widthScale + 1; ++i)
{
    x = 0;
    for (j = 0; j < lengthScale + 1; ++j)
    {
        coords[nodeId * 3] = x; // x
        coords[nodeId * 3 + 1] = y; // y
    }
}

```

```

        coords[nodeId * 3 + 2] = z; // z
        nodeLabels[nodeStartIndex + nodeId] = nodeLabel++;
        x = x + currentXRadian;
        ++nodeId;
    }
    y = y + currentYRadian;
}
coordContainer.Append(coords.data(), totalNodeCount * 3);

// 单元数据
int numElement = widthScale * lengthScale;
int* connectivity = (int*)malloc(sizeof(int) * numElement * 4);
int* elementLabels = (int*)malloc(sizeof(int) * numElement);
int* connectivityArr = connectivity;
int* elementLabelArr = elementLabels;
for (i = 0; i < widthScale; ++i)
{
    for (j = 0; j < lengthScale; ++j)
    {
        *connectivityArr++ = i * (lengthScale + 1) + j + nodeStartIndex;
        *connectivityArr++ = i * (lengthScale + 1) + j + nodeStartIndex + 1;
        *connectivityArr++ = (i + 1) * (lengthScale + 1) + j + nodeStartIndex + 1;
        *connectivityArr++ = (i + 1) * (lengthScale + 1) + j + nodeStartIndex;
        *elementLabelArr++ = elemLabel++;
    }
}

QString S4ElementType = "S4R";
classes[numClasses] = bmeElementClass::ConstructObject(numElement, S4ElementType,
connectivity);
classes[numClasses]->SetUserElementLabel(elementLabels);
numClasses++;

bmeMesh* mesh = new omeMesh(1000u, nodeStartIndex + totalNodeCount, coordContainer,
numClasses, classes);
mesh->GetNodeData().SetUserNodeLabels(nodeLabels.data());
mesSetMesh(*fl, 1000u, mesh);

ftrPrimaryObjShortcut sc = PartShortcut();
samModelDrawableImplBase* modelDrawable =
samMdbDrawable::getDrawable()->getModel(sc.ModelShortcut().Name());
if (modelDrawable)
{
    samPartDrawableImplBase* partDrawable = modelDrawable->getPart(sc.Name());
    if (partDrawable)
        partDrawable->setOutOfDate(partDispMeshTS);
}

```



```
}
```

```
return 0;
```

```
}
```

3.1.7 使用 cmake 构建工程

上述操作执行完毕后，使用 **cmake** 工具构建工程，输出目录为工程目录下的 **build** 目录。

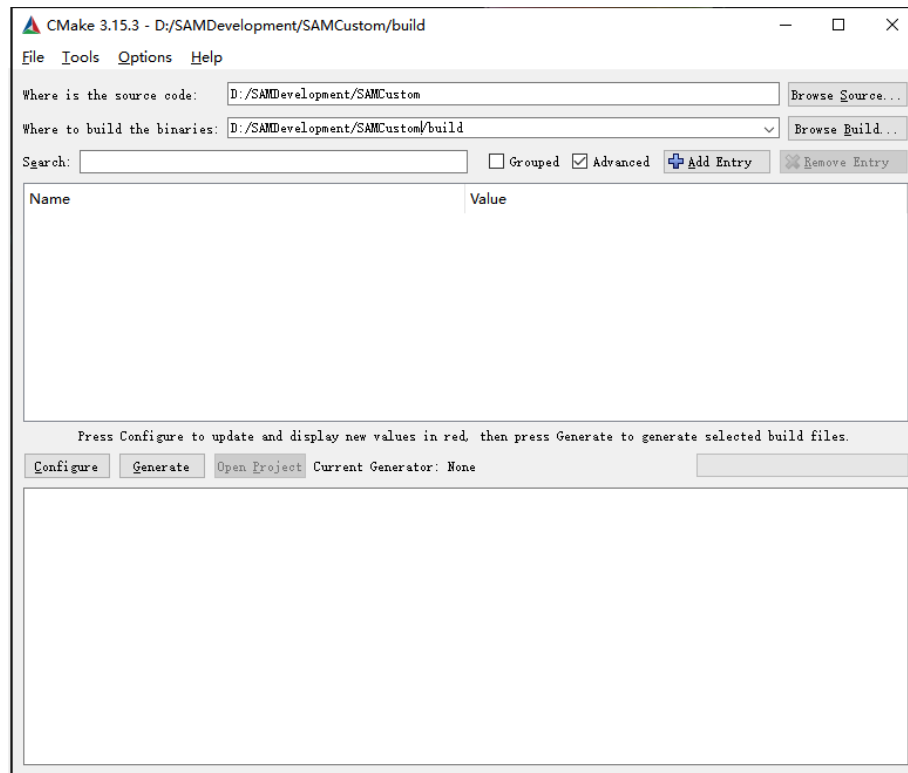


图 39 cmake 构建工程

点击 **Configure**，弹出新建文件夹提示，点击 **Yes**。

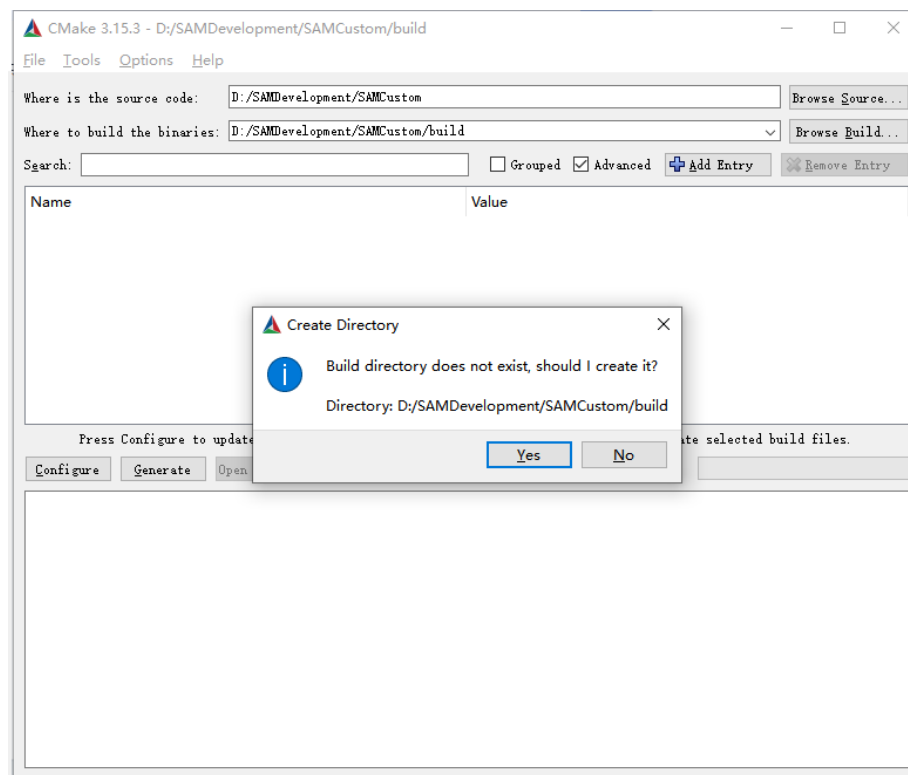


图 40 cmake 构建工程

在弹出的窗口中选择编译器为 VS2015，生成平台版本为 x64

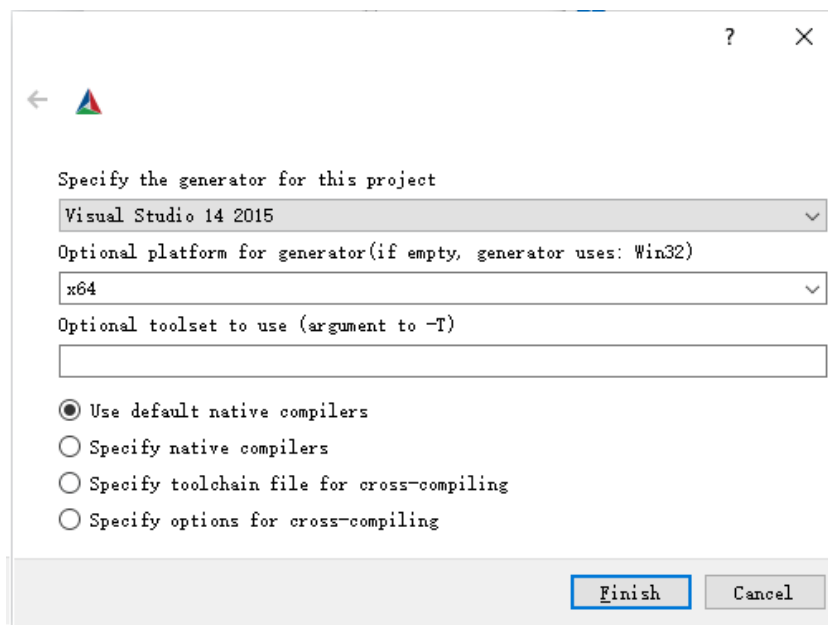


图 41 选择编译器与平台版本

点击 Finish，点击 Configure 进行配置，等待配置完成。

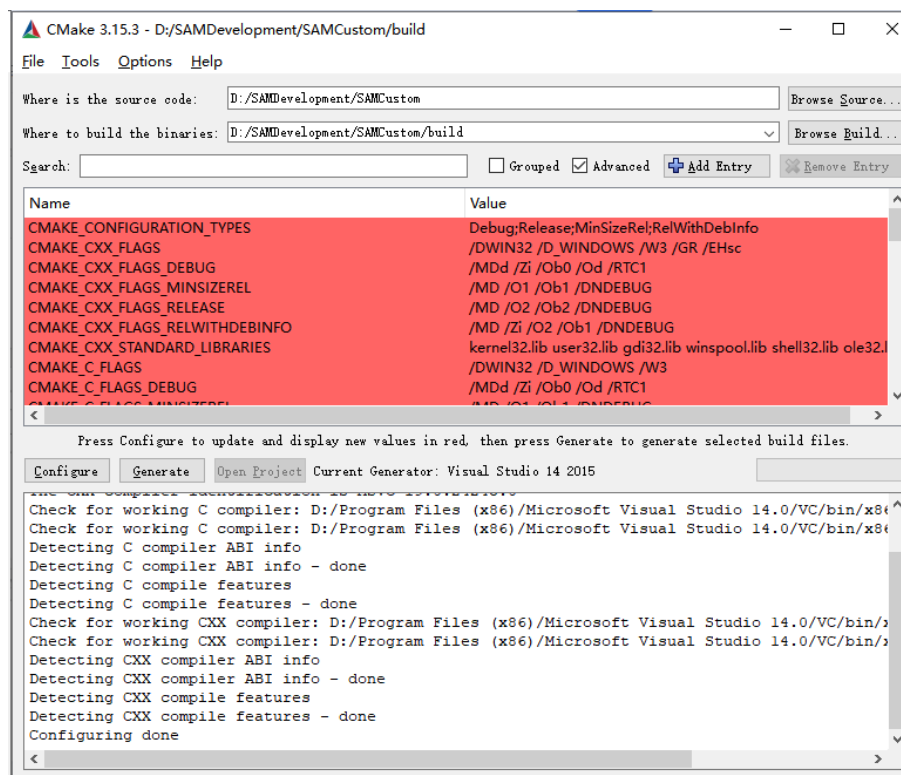


图 42 配置工程

配置完成后，点击 **Generate**，等待生成完成后，即可打开工程。

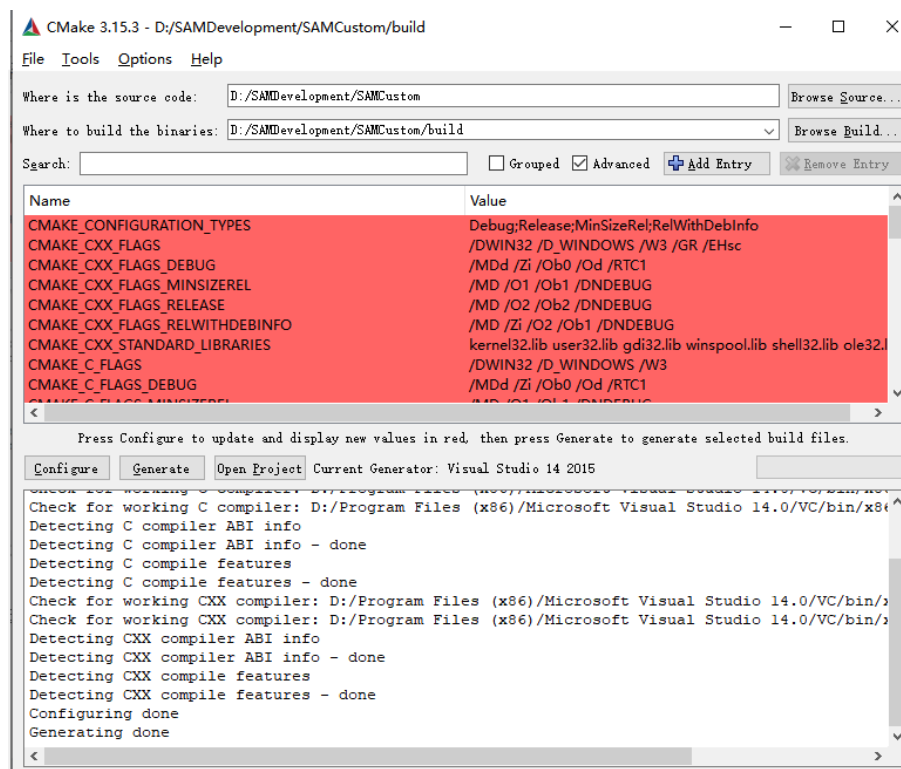


图 43 生成工程

3.2 生成动态库及使用

3.2.1 动态库生成

打开工程，完成代码编写后，使用编译器开始生成，成功后在输出文件目录下会生成一个.pyd 动态库，此处我们生成的是与项目同名的 Custom.pyd。

3.2.2 使用动态库

要在 SAM 中使用动态库，首先我们需要把生成的动态库 Custom.pyd 拷贝到 SAM 启动项目录。

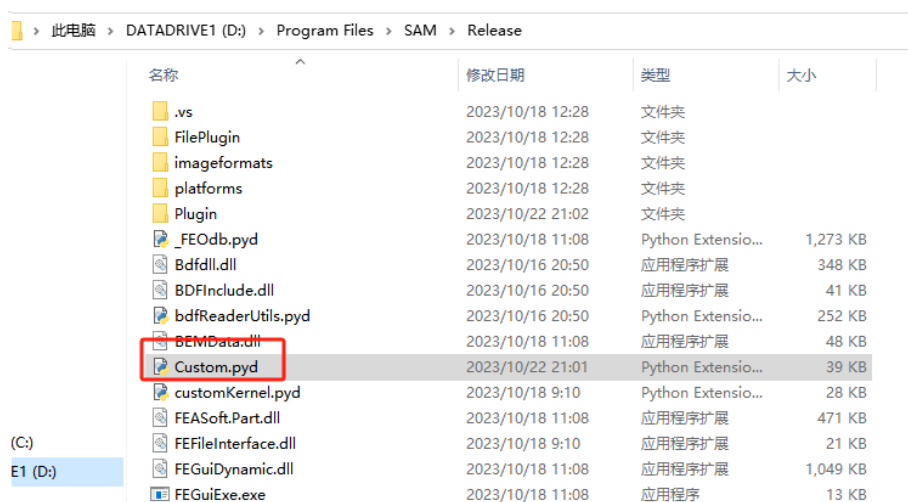


图 44 拷贝动态库到启动项目录

然后需要在界面模块中，对该启动项进行注册和初始化。

如图所示，编辑我们自定义的模块脚本（软件启动项目录下 Plugin/CustomModuleGui.py），在模块类中添加一行代码：

`self.setKernelInitialCommand("import Custom")`

此代码含义为发送初始化命令"import Custom"，在 SAM 软件启动时便可加载我们生成的动态库 Custom.pyd。

```
class CustomModuleGui(FEModuleGui):
    def __init__(self):
        FEModuleGui.__init__(self, "CustomModule", FEModuleGui.PART)
        self.registerToolset(customToolset, GUI_IN_MENUBAR)
        self.setKernelInitialCommand("import Custom")

customModule = CustomModuleGui()
```

图 45 添加初始化命令

然后我们可以在模块菜单栏中新增一个按钮，用来发送动态库中设置好的命令语句。

```
class CustomToolsetGui(FEToolsetGui):  
    def __init__(self):  
        FEToolsetGui.__init__(self, "Custom")  
        menu = QMenu("Custom", self)  
        menu.addAction("SayHello...", self.SayHello)  
        menu.addAction("Create Rectangle", self.CreateRectangle)  
  
    @Slot()  
    def CreateRectangle(self):  
        cmd = "p.CreateRectangle(5, 10, 5, 10)"  
        FEClient.instance().sendExecuteCommand(cmd)
```

图 46 新增按钮以发送命令

保存文件后，双击 SAM 启动项目目录下批处理脚本 samAPP.bat，打开软件。

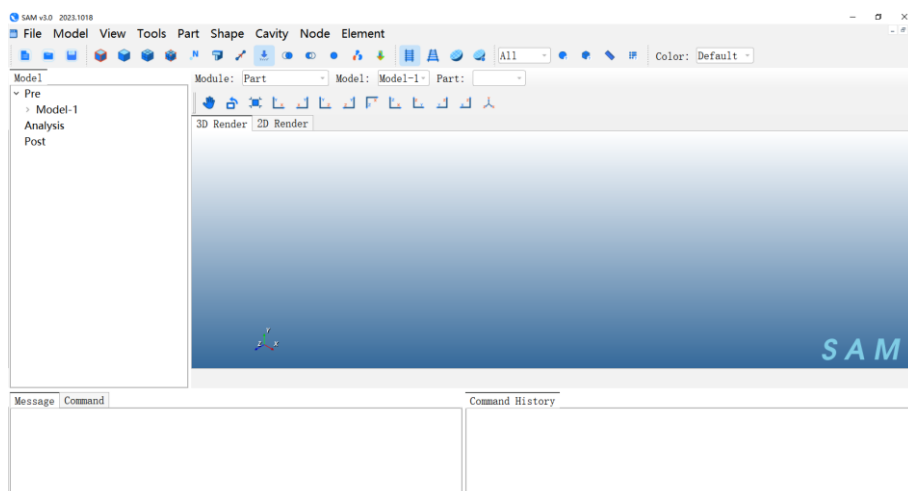


图 47 打开软件

点击 Part->Create，打开 Part 创建窗口，点击 OK，完成 Part 的创建。

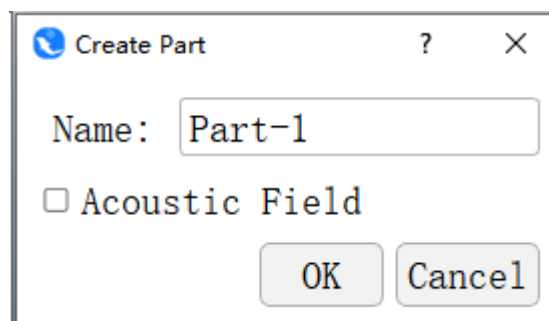


图 48 创建 Part

然后在 Module 下拉框中，切换到我们创建的 CustomModule 模块。

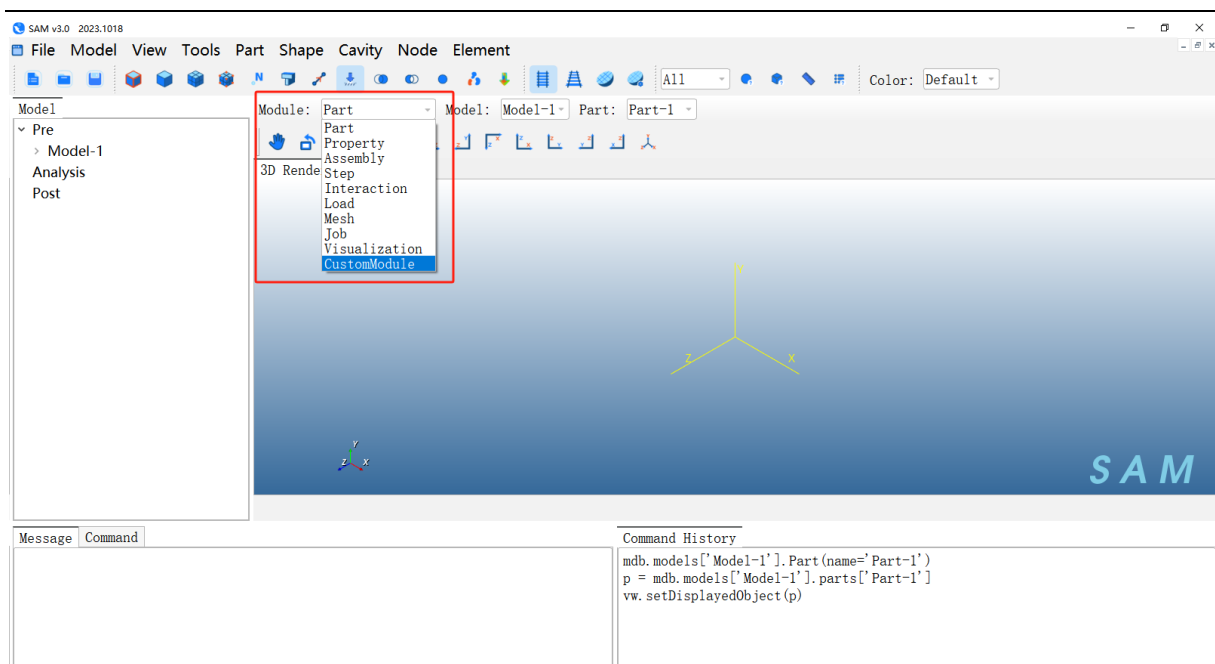


图 49 创建 Part

点击 Custom->Create Rectangle，即可发送我们设置好的命令，可以看到界面上创建了一个矩形网格模型。

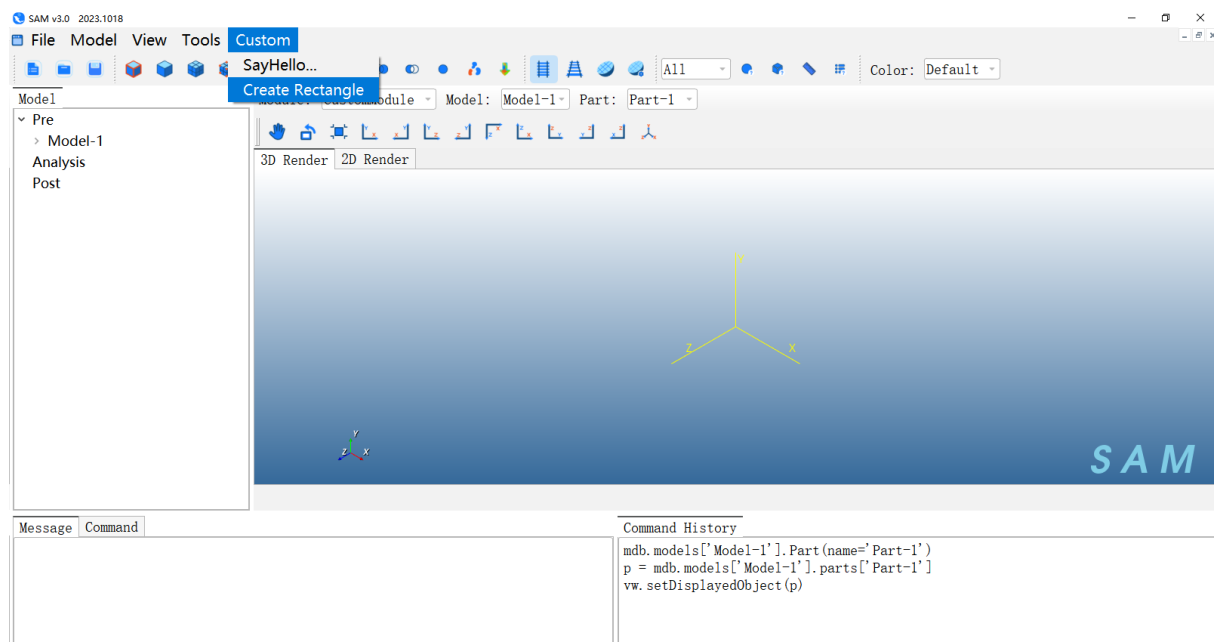


图 50 点击按钮

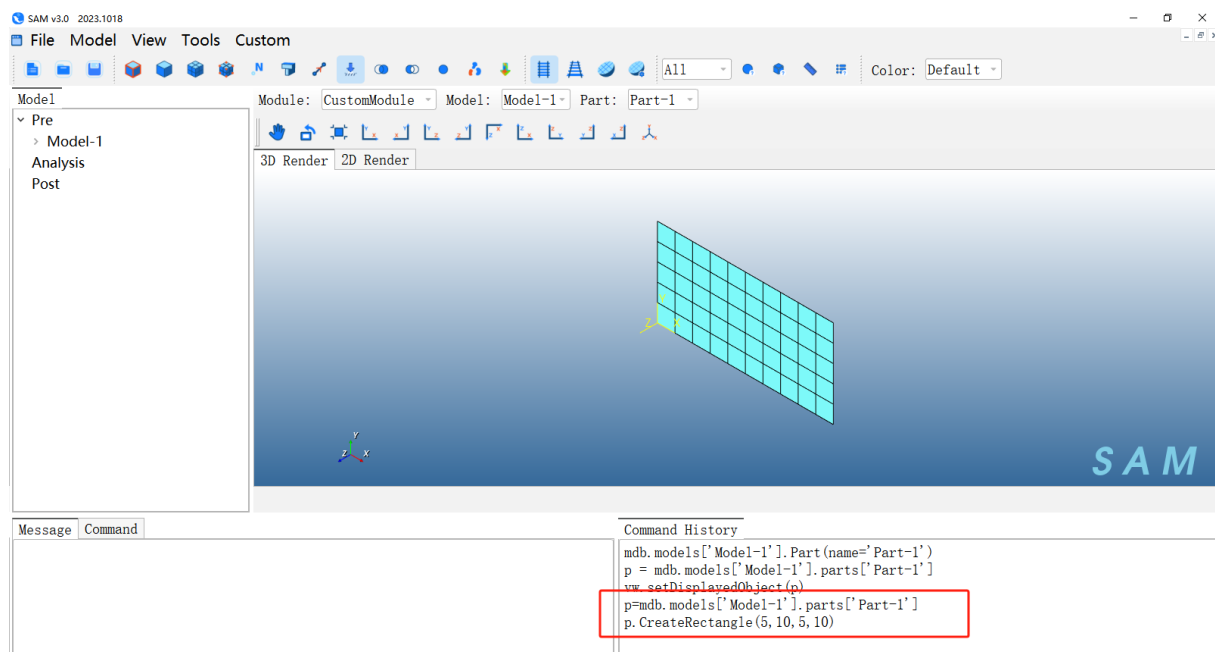


图 51 发送命令，创建网格模型

3.3 C++函数说明

- 分级目录说明：
- 3 级目录为模块
- 4 级目录为类名
- 5 级目录为类下的函数名

3.3.1 基础数据类型

3.3.1.1 cow2DArray

二维数组

3.3.1.2 cowList

数组列表

3.3.1.3 cowMap

map 容器

3.3.2 共性基础类

3.3.2.1 gdyCompositeHitSet

用于描述选取区域的类，可以设置选取区域数据对应的类型。

3.3.2.1.1 AddBitVecHit

作用描述	增加选取区域
参数输入说明	Context 选取区域类型，bitvec 选取区域数据
参数输出说明	无
API 实现	<code>void AddBitVecHit(gdyCompositeHitCTX context, const omiBlkBitVect& bitvec)</code>

3.3.2.2 gdyCompositeHit

用于描述选取类型

3.3.2.2.1 gdyCompositeHitCTX

作用描述	设置选取类型
参数输入说明	<code>gdyCompositeType</code> 拾取类型， <code>gdyHitRecType</code> 拓扑类型， <code>gdyCompHitType</code> (节

	点/单元/面/边)， gdySurfType（选取第几个面或者面的正反方向）， instanceID(特征 ID)
参数输出说明	无
API 实现	<pre>gdyCompositeHitCTX(gdyCompositeType comtype, gdyHitRecType hitrectype, gdyCompHitType hittype, gdySurfType surftype, uint inst_id)</pre>

3.3.2.3 omiBlkBitVect

3.3.2.3.1 omiBlkBitVect

作用描述	构造数据对象（用于存储，选取区域的节点/单元等相关数据）
参数输入说明	无
参数输出说明	无
API 实现	omiBlkBitVect()

3.3.2.3.2 SetBit

作用描述	设置选取区域数据的 Index
参数输入说明	Index(节点/单元等对象的 Index)
参数输出说明	无
API 实现	void SetBit(uint index);

3.3.2.4 rgnFSet

集合对象类，描述集合的类型，集合所包含的数据。以及集合相关操作函数。

3.3.2.4.1 rgnFSet

作用描述	构造 set 对象
参数输入说明	pFList（特征列表）， scName（set 名称）
参数输出说明	无
API 实现	rgnFSet(ftrFeatureList* pFList, const QString& scName)

3.3.2.4.2 SetSpace

作用描述	构造 set 网格数据
参数输入说明	gdyCompositeHitSet（选取区域）
参数输出说明	无
API 实现	rgnFMeshSetData(const gdyCompositeHitSet&, rgnCDimBits dim = rgnC_THREE_DIM)

3.3.2.4.3 SetMeshSetData

作用描述	设置 set 网格数据
参数输入说明	rgnFMeshSetData (set 网格数据对象)
参数输出说明	无
API 实现	void SetMeshSetData(rgnFMeshSetData* = 0);

3.3.2.5 RgnKRegion

对 rgnFSet 进行包装，能够快速访问 set 对象以及对应的数据（节点、单元、几何等数据）。

3.3.2.5.1 rgnKRegion

作用描述	构造区域对象
参数输入说明	flist (特征列表)，owner_name (region 所属对象名称，例如 part 或者 assembly)，set ()
参数输出说明	无
API 实现	rgnKRegion(const ftrFeatureList& flist, const QString& owner_name, const rgnFSet* set);

3.3.2.6 RgnCRegion

集合对象的描述，通过此对象，能够获取 set 的类型，名称等。

3.3.2.7 RgnFMeshSetData

3.3.2.7.1 rgnFMeshSetData

作用描述	构造 set 网格数据
参数输入说明	gdyCompositeHitSet (选取区域)
参数输出说明	无
API 实现	rgnFMeshSetData(const gdyCompositeHitSet&, rgnCDimBits dim = rgnC_THREE_DIM)

3.3.2.8 asoFUtils

该类处理装配下相关的操作，支持创建/获取 instance，更新网格数据等。

3.3.2.8.1 CreateInstance

作用描述	创建 instance
参数输入说明	featList (特征列表)，scInstName (instance 的名称)，scModelName (model 的名称)，part (part 对象)，partRepos(part 容器)，xForm (instance 的空间变换值)

参数输出说明		basBasis 对象
API 实现		<pre>static asoFPartInstance* CreateInstance(ftrFeatureList & featList, const QString & scInstName, const QString & scModelName, const ptoKPart & part, const ptoKPartRepository & partRepos, const gslMatrix & xForm, bool qLightWeight, bool qRegenConsts, bool qFireQuery =true);</pre>

3.3.3 Mdb

3.3.3.1 basBasis

初始模型。基于该类可以操作 mdb。

3.3.3.1.1 Instance

作用描述	构造了一个 mdb 对象，并在 mdb 下构造了一个默认的 model 对象
参数输入说明	无
参数输出说明	basBasis 对象
API 实现	Static basBasis* Instance()

3.3.3.1.2 Fetch

作用描述	获取 basMdb 对象
参数输入说明	无
参数输出说明	basMdb 对象
API 实现	const basMdb& Fetch()

3.3.3.1.3 UncheckedReplace

作用描述	更新数据
参数输入说明	basMdb 对象

参数输出说明	无
API 实现	<code>void UncheckedReplace(const basMdb& root)</code>

3.3.3.1.4 Replace

作用描述	更新数据
参数输入说明	basMdb 对象
参数输出说明	无
API 实现	<code>void Replace(const basMdb& root)</code>

3.3.3.2 basMdb

mdb 类，能够设置和获取 model 和 job。

3.3.3.2.1 GetModels

作用描述	获取 basModelMap 对象
参数输入说明	无
参数输出说明	basModelMap 对象
API 实现	<code>inline basModelMap& GetModels()</code>

3.3.4 Model

3.3.4.1 basNewModel

model 对象，能够获取 model 下所的数据容器对象，比如 parts,materials 等

3.3.4.1.1 basNewModel

作用描述	构造 basNewModel 对象
参数输入说明	无
参数输出说明	无
API 实现	<code>basNewModel()</code>

3.3.4.1.2 Rewire

作用描述	basNewModel 对象写入 model 的名称
参数输入说明	无
参数输出说明	无
API 实现	<code>void Rewire(const QString &modelName)</code>

3.3.4.1.3 GetPart/ConstGetPart

作用描述	获取所有型材
参数输入说明	无
参数输出说明	kbpCBeamProfileContainer 材料容器对象
API 实现	<code>GetBeamProfiles()</code>

3.3.4.1.4 GetMaterials/ ConstGetMaterials

作用描述	获取所有材料
参数输入说明	无
参数输出说明	kmaCMaterialContainer 材料容器对象
API 实现	GetMaterials()

3.3.4.1.5 GetBeamProfiles/ ConstGetBeamProfiles

作用描述	获取所有型材
参数输入说明	无
参数输出说明	kbpCBeamProfileContainer 型材容器对象
API 实现	GetBeamProfiles()

3.3.4.1.6 GetSections/ ConstGetSections

作用描述	获取所有型材
参数输入说明	无
参数输出说明	ksecSectionContainer 属性容器对象
API 实现	GetBeamProfiles()

3.3.4.1.7 GetAssy/ ConstGetAssy

作用描述	获取所有装配
参数输入说明	无
参数输出说明	asoKAssemblyRepository 装配容器对象
API 实现	GetAssy()/ConstGetAssy()

3.3.4.1.8 GetSteps/ ConstGetSteps

作用描述	获取所有 steps
参数输入说明	无
参数输出说明	kprCStepContainerstep 容器对象
API 实现	GetSteps()

3.3.4.1.9 GetBCs/ ConstGetBCs

作用描述	获取所有约束
参数输入说明	无
参数输出说明	kbcCBCContainer 约束容器对象
API 实现	GetBCs()

3.3.4.1.10 GetLoads/ ConstGetLoads

作用描述	获取所有载荷
参数输入说明	无
参数输出说明	kloCLoadContainer 载荷容器对象

API 实现	GetLoads()
--------	------------

3.3.4.1.11 GetDiscreteFields/ ConstGetDiscreteFields

作用描述	获取所有离散场
参数输入说明	无
参数输出说明	kfdCDiscreteFieldContainer 离散场容器对象
API 实现	GetDiscreteFields()

3.3.4.1.12 GetConstraints/ConstGetConstraints

作用描述	获取所有约束条件（MPC）
参数输入说明	无
参数输出说明	kcoCConstraintContainer 约束条件容器对象
API 实现	GetConstraints()

3.3.4.2 basModelMap

map 表，能够快速访问 model。

3.3.4.2.1 Insert

作用描述	插入对象数据
参数输入说明	modelName(模型名称), basNewModel 对象
参数输出说明	无
API 实现	bool Insert(const QString &modelName, const basNewMode &newModel)

3.3.4.2.2 Get

作用描述	获取名称对应 model 对象
参数输入说明	modelName
参数输出说明	basNewModel 对象
API 实现	basNewModel & Get(const QString&)

3.3.5 Part

Part 类，基于该类，可以设置 part 的参数以及获取 part 下相关的属性。比如 part 所对应的属性，part 的特征列表等。

3.3.5.1 ptoKPart

3.3.5.1.1 ptoKPart

作用描述	构建 part 对象
参数输入说明	theory (part 对象描述), name (part 的名称)
参数输出说明	无
API 实现	ptoKPart(bdoTheory theory, const QString& name)

3.3.5.1.2 SetId

作用描述	设置 part 对象的 ID
参数输入说明	iId(id 值)
参数输出说明	无
API 实现	void SetId(uint iId)

3.3.5.1.3 GetFeatureList

作用描述	获取特征列表
参数输入说明	无
参数输出说明	ftrFeatureList (特征列表)
API 实现	ftrFeatureList* GetFeatureList()

3.3.5.1.4 GetSectionAssignments

作用描述	获取特征列表
参数输入说明	无
参数输出说明	ptoCPartAttributeList (属性列表)
API 实现	virtual ptoCPartAttributeList& GetSectionAssignments();

3.3.5.1.5 GetEngineeringFeatures()/ConstGetEngineeringFeatures()

作用描述	获取工程特征列表
参数输入说明	无
参数输出说明	kefCEngineeringFeatures (属性列表)
API 实现	kefCEngineeringFeatures& GetEngineeringFeatures();

3.3.5.2 ptoKPartRepository

part 的容器对象，能够增加、删除 part。

3.3.5.2.1 ptoKGetPartRepos

作用描述	获取 model 下的所有 part 对象
参数输入说明	theory (part 对象描述), name (part 的名称)

参数输出说明	无
API 实现	<code>ptoKPart(bdoTheory theory, const QString& name)</code>

3.3.5.2.2 Insert

作用描述	将 part 对象插入到容器 parts 中
参数输入说明	partName（部件名称），part（部件对象）
参数输出说明	Bool 是否成功
API 实现	<code>bool Insert(const Key&, const Value&)</code>

3.3.5.2.3 IsMember

作用描述	判断 part 容器中是否有
参数输入说明	partName（部件名称）
参数输出说明	Bool 是否成功
API 实现	<code>bool IsMember(const Key&) const;</code>

3.3.5.2.4 Get

作用描述	获取名称对应 part 对象
参数输入说明	partName（部件名称）
参数输出说明	<code>ptoKPart</code> 对象
API 实现	<code>ptoKPart& Get(const QString&)</code>

3.3.6 Material

3.3.6.1 kmaCMaterialContainer

材料容器对象，能够增加、删除材料。

3.3.6.1.1 Insert

作用描述	材料容器对象中插入材料对象
参数输入说明	materialName（材料名称）， <code>kmaCMaterial</code> （材料对象）
参数输出说明	Bool 是否成功
API 实现	<code>bool Insert(const QString&, kmaCMaterial*)</code>

3.3.6.2 kmaCMaterial

材料类，定义了材料包含的属性，比如密度、杨氏模量、泊松比、塑性等。

其中成员变量 `kmaCDensityCOW density` 用于保存材料密度，`kmaCElasticCOW elastic` 用于保存杨氏模量和泊松比。

通过 `kmaCMaterial()` 实例化材料类后，需要对成员变量 `kmaCDensityCOW density`、`kmaCElasticCOW elastic` 进行赋值；

3.3.6.2.1 kmaCMaterial

作用描述	构造材料对象
参数输入说明	无
参数输出说明	无
API 实现	<code>kmaCMaterial()</code>

3.3.6.2.2 kmaCDensity

材料密度类，在成员变量 `kmaCMaterialTable table` 中保存材料密度值。

通过 `kmaCMaterial()`实例化材料密度类后，需要对成员变量 `kmaCMaterialTable table` 进行赋值。

作用描述	构造密度对象
参数输入说明	无
参数输出说明	无
API 实现	<code>kmaCMaterial()</code>

3.3.6.2.3 kmaCMaterialTable

作用描述	构造数据列表
参数输入说明	Row（行），col（列）
参数输出说明	无
API 实现	<code>kmaCMaterialTable(int row, int col)</code>

3.3.6.2.4 kmaCElastic

材料的弹性属性类，在成员变量 `kmaCMaterialTable table` 中保存杨氏模量和泊松比。

通过 `kmaCElastic ()`实例化材料密度类后，需要对成员变量 `kmaCMaterialTable table` 进行赋值。

作用描述	构造弹性属性对象
参数输入说明	无
参数输出说明	无
API 实现	<code>kmaCElastic ()</code>

3.3.7 Profile

3.3.7.1 kbpCBeamProfileContainer

型材容器对象，能够增加、删除型材。

3.3.7.1.1 Insert

作用描述	型材容器对象中插入型材对象
参数输入说明	profileName（型材名称），profile（型材对象）
参数输出说明	Bool 是否成功
API 实现	bool Insert (const QString&, kbpCTbeamProfile*)

3.3.7.2 kbpCGeneralizedProfile

Generalized 型材类，包含成员变量 double area、i11、i12、i22、j，实例化后，需要对上述成员变量进行赋值；

作用描述	构造 Generalized 型材
参数输入说明	无
参数输出说明	无
API 实现	kbpCGeneralizedProfile()

3.3.7.3 kbpCTbeamProfile

T 型材类，包含成员变量 double b、h、tf、tw，实例化后，需要对上述成员变量进行赋值；

作用描述	构造 T 型材
参数输入说明	无
参数输出说明	无
API 实现	kbpCTbeamProfile ()

3.3.7.4 kbpCLbeamProfile

L 型材类，包含成员变量 double a、b、t1、t2，实例化后，需要对上述成员变量进行赋值；

作用描述	构造 L 型材
参数输入说明	无
参数输出说明	无
API 实现	kbpCLbeamProfile ()

3.3.7.5 kbpCRectangularbeamProfile

扁钢型材类，包含成员变量 double a、b，实例化后，需要对上述成员变量进行赋值；

作用描述	构造 Rectangular 型材
参数输入说明	无
参数输出说明	无
API 实现	kbpCRectangularbeamProfile ()

3.3.7.6 kbpCCircularProfile

圆型材类，包含成员变量 **double r**，实例化后，需要对上述成员变量进行赋值；

作用描述	构造 Circular 型材
参数输入说明	无
参数输出说明	无
API 实现	<code>kbpCCircularProfile ()</code>

3.3.7.7 kbpCPipeProfile

管型材类，包含成员变量 **double r、t**，实例化后，需要对上述成员变量进行赋值；

作用描述	构造 Pipe 型材
参数输入说明	无
参数输出说明	无
API 实现	<code>kbpCPipeProfile ()</code>

3.3.7.8 kbpCIbeamProfile

工字形型材类，包含成员变量 **double b、h、tf、tw、l**，实例化后，需要对上述成员变量进行赋值；

作用描述	构造 I 型材
参数输入说明	无
参数输出说明	无
API 实现	<code>kbpCIbeamProfile ()</code>

3.3.7.9 kbpCBoxProfile

BOX 型材类，包含成员变量 **double a、b、t1、t2、t3、t4**，实例化后，需要对上述成员变量进行赋值；

作用描述	构造 Box 型材
参数输入说明	无
参数输出说明	无
API 实现	<code>kbpCBoxProfile ()</code>

3.3.8 Section

3.3.8.1 kseCSectionContainer

属性容器对象，能够添加、删除属性。

3.3.8.1.1 Insert

作用描述	增加属性
参数输入说明	<code>sectionName</code> (模型名称)， <code>section</code> 属性对象

参数输出说明	无
API 实现	<code>bool Insert(const QString&, ITEM*)</code>

3.3.8.2 kseCBeamSection

梁属性类，包含成员变量 `QString profile`、`QString material`、`IntegrationEnm integration` 等，如果是 `Generalized` 型材，需要设置 `integration = BEFORE_ANALYSIS`，并对成员变量 `QString profile`（型材名）、`clsxp2DArrayDouble table`（杨氏模量和泊松比）、`double density`（密度）进行赋值；如果是其他型材，则 `integration = DURING_ANALYSIS`，需要对成员变量 `QString profile`、`QString material` 进行赋值；

作用描述	构造梁属性
参数输入说明	无
参数输出说明	无
API 实现	<code>kseCBeamSection ()</code>

3.3.8.3 kseCTrussSection

杆属性类，包含成员变量 `QString material`、`double area`，实例化后需要对上述成员变量进行赋值；

作用描述	构造杆属性
参数输入说明	无
参数输出说明	无
API 实现	<code>kseCTrussSection ()</code>

3.3.8.4 kseCHomogenousShellSection

壳属性类，包含成员变量 `QString material`、`double thickness`，实例化后需要对上述成员变量进行赋值；

作用描述	构造壳属性
参数输入说明	无
参数输出说明	无
API 实现	<code>kseCHomogenousShellSection ()</code>

3.3.8.5 kseCHomogenousSolidSection

体属性类，包含成员变量 `QString material`，实例化后需要对上述成员变量进行赋值；

作用描述	构造体属性
参数输入说明	无
参数输出说明	无
API 实现	<code>kseCHomogenousSolidSection ()</code>

3.3.9 Assignment

3.3.9.1 ptoCSectionAssignment

物理属性分配的类，该类定义了物理属性以及物理属性所对应的区域。

3.3.9.1.1 ptoCSectionAssignment

作用描述	构造物理属性分配
参数输入说明	<code>rgnCRegion (cRegion 对象)</code> ， <code>sectionName (属性名称)</code>
参数输出说明	无
API 实现	<code>ptoCSectionAssignment(const rgnCRegion&, const QString& sectionName, double offset = 0)</code>

该类能够设置梁的方向 `n1`，对应为成员变量 `gslPersistentVector n1`，如果这批梁单元的 `n1` 都相同，则 `n1type = UNIFORM`，如果不同，则需要创建离散场 `kfdCDiscreteField`，用于存储单元 ID、单元 `n1` 的对应关系，`n1type = FIELD`、`n1field = fieldName`

该类也能够设置梁的偏置，对应为成员变量 `gslPersistentVector offset1Vector`、`offset2Vector`，赋值方式和 `n1` 类似。

3.3.10 Assembly

3.3.10.1 asoKAssembly

装配对象，能够构造装配对象，获取装配下的特征等。

3.3.10.1.1 GetFeatureList

作用描述	获取特征列表
参数输入说明	无
参数输出说明	<code>ftrFeatureList (装配特征列表)</code>
API 实现	<code>ftrFeatureList* GetFeatureList ()</code>

3.3.10.2 asoFUtils

3.3.10.2.1 CreateInstance

作用描述	创建 Instance
参数输入说明	ftFeatureList(特征列表), scInstName(Instance 名称), scModelName(模型名称), ptokPart (Part 对象), ptokPartRepository (Part 容器), gslMatrix (gslMatrix 对象)
参数输出说明	asoFPartInstance (装配特征)
API 实现	<pre>static asoFPartInstance* CreateInstance(ftFeatureList& featList, const QString& scInstName, const QString& scModelName, const ptokPart& part, const ptokPartRepository& partRepos, const gslMatrix& xForm, bool qLightWeight, bool qRegenConsts, bool qFireQuery =true);</pre>

3.3.11 Constraint

3.3.11.1 kcoCConstraintContainer

存储 model 下的所有 Constraint 对象，能够添加，删除 Constraint。

3.3.11.1.1 Insert

作用描述	插入一个 Constraint
参数输入说明	constraints 名称, Constraint 对象,
参数输出说明	Bool 是否成功
API 实现	<pre>bool Insert(const QString&, const kcoCConstraint&)</pre>

3.3.11.2 kcoCCoupling

kcoCConstraint 的子类，用于定义 MPC，包含成员变量 rgnCRegion refPoint 、 surface，保存 MPC 的主节点和从节点；kcoCCouplingTypeEnm couplingType，用于定义 MPC 类型，包括 RBE2、RBE3 等；bool u1~u6，用于定义 MPC 的自由度。

作用描述	构造 kcoCCoupling
参数输入说明	无
参数输出说明	无
API 实现	<pre>kcoCCoupling()</pre>

3.3.12 Spring

3.3.12.1 kefCSpring

kefCSpring 类，用于保存弹簧属性，包含成员变量 **rgnCRegion region**，用于存储弹簧节点作用域；

3.3.12.2 kefCSpringContainer

kefCSpring 容器，存储 **Instance** 下的所有 **kefCSpring** 对象，能够添加，删除 **kefCSpring**。

3.3.12.2.1 Insert

作用描述	弹簧容器对象中插入弹簧对象
参数输入说明	springName （弹簧名称）， kefCSpring （弹簧对象）
参数输出说明	Bool 是否成功
API 实现	<code>bool Insert(const QString&, kefCSpring&)</code>

3.3.12.3 kefCSpringToGround

kefCSpring 子类，用于定义接地弹簧，包含成员变量 **int dof1**，用于保存弹簧自由度；**double springStiffness**，用于保存弹簧刚度；**double dashpotCoeff**，用于保存阻尼系数；

作用描述	构造接地弹簧
参数输入说明	无
参数输出说明	无
API 实现	<code>kefCSpringToGround()</code>

3.3.12.4 kefCTwoPointSpring

kefCSpring 子类，用于定义两节点弹簧，包含成员变量 **int dof1、dof2**，用于保存弹簧节点 1、2 自由度；**double springStiffness**，用于保存弹簧刚度；**double dashpotCoeff**，用于保存阻尼系数；**rgnCRegionPairArray regionPairs** 保存节点 1、2 的 **rgnCRegion**。

作用描述	构造两节点弹簧
参数输入说明	无
参数输出说明	无
API 实现	<code>kefCTwoPointSpring()</code>

3.3.13 Inertia

3.3.13.1 kefCIInertia

kefCIInertia 类，用于保存惯性相关数据；

3.3.13.2 kefCIInertiaContainer

kefCIInertia 容器，存储 Instance 下的所有 kefCIInertia 对象，能够添加，删除 kefCIInertia。

3.3.13.2.1 Insert

作用描述	惯性容器对象中插入惯性对象
参数输入说明	InertiaName（惯性名称），kefCIInertia（惯性对象）
参数输出说明	Bool 是否成功
API 实现	bool Insert(const QString&, kefCIInertia&)

3.3.13.3 kefCPointMassInertia

kefCIInertia 子类，用于定义接地质量点，包含成员变量 double mass，保存质量；
rgnCRegion region，用于存储质量点作用域；

作用描述	构造质量点
参数输入说明	无
参数输出说明	无
API 实现	kefCPointMassInertia()

3.3.14 Step

3.3.14.1 kprCStepContainer

存储 model 下的所有 step 对象，能够添加，删除 step。

3.3.14.1.1 Insert

作用描述	插入一个 step
参数输入说明	step 名称，step 对象，
参数输出说明	Bool 是否成功
API 实现	bool Insert(const QString&, const kprCStep&)

3.3.14.1.2 InsertAfter

作用描述	在指定名称 Step 后面插入一个 step
参数输入说明	之前的 step 名称, 要插入的 step 名称, 要插入的 step 对象,
参数输出说明	Bool 是否成功
API 实现	<code>bool InsertAfter(const QString&, const QString&, const kprCAnalysisStep&, bool noSequencer = false)</code>

3.3.14.1.3 Remove

作用描述	删除指定的 step
参数输入说明	stepName (Step 的名称)
参数输出说明	Bool 是否成功
API 实现	<code>bool Remove(const QString&)</code>

3.3.14.2 KprCStepNameProcedurePair

此类描述了 **step** 名称和 **step** 对象的对应关系, 并且构建了一个 **list** 用于存储该对象。

3.3.14.3 kprCStaticStep

静力分析步, 次类包含了静力分析步的构造函数, 相关参数以及函数方法。

作用描述	构造静力分析步构造函数
参数输入说明	无
参数输出说明	无
API 实现	<code>kprCStaticStep()</code>

3.3.14.4 kprCStaticLinearPerturbationStep

线性静力分析步, 次类包含了线性静力分析步的构造函数, 相关参数以及函数方法。
包含成员变量 **double minEigen**、**maxEigen**, 保存最小最大特征值; **int numEigen** 表示特征值个数; **NormalizationEnm normalization** 保存特征向量归一化方法;

3.3.14.4.1 kprCStaticLinearPerturbationStep

作用描述	构造线性静力分析步构造函数
参数输入说明	无
参数输出说明	无
API 实现	<code>kprCStaticLinearPerturbationStep()</code>

3.3.14.5 kprCFrequencyStep

模态分析步, 次类包含了模态分析步的构造函数, 相关参数以及函数方法。

3.3.14.5.1 kprCFrequencyStep

作用描述	构造模态分析步构造函数
参数输入说明	无
参数输出说明	无
API 实现	<code>kprCFrequencyStep ()</code>

3.3.14.6 kprCSteadyStateModalStep

稳态分析步，次类包含了稳态分析步的构造函数，相关参数以及函数方法。

3.3.14.6.1 kprCSteadyStateModalStep

作用描述	稳态分析步构造函数
参数输入说明	无
参数输出说明	无
API 实现	<code>kprCSteadyStateModalStep()</code>

3.3.15 BC

3.3.15.1 kbcCBCContainer

存储 model 下的所有 bc 对象，能够添加，删除 bc。

3.3.15.1.1 UpdateSteps

作用描述	更新 step 和 BC 的状态关系
参数输入说明	
参数输出说明	无
API 实现	<code>void UpdateSteps(const kprCStepNameProcedurePairList&,const QString&,bool)</code>

3.3.15.1.2 Insert

作用描述	插入一个 step
参数输入说明	step 名称，step 对象，
参数输出说明	Bool 是否成功
API 实现	<code>bool Insert(const QString&, const kprCStep&)</code>

3.3.15.1.3 Remove

作用描述	删除指定的 step
参数输入说明	stepName(Step 的名称)
参数输出说明	Bool 是否成功

API 实现	<code>bool Remove(const QString&)</code>
--------	--

3.3.15.2 kbcCConnDisplacementBC

位移约束，包含其相关的参数及函数。包含成员变量 `rgnCRegion region`，用于保存约束节点作用域；`DistribTypeEnum distribution`，用于判断约束是否均匀分布；

3.3.15.2.1 Insert

作用描述	插入指定 step 的位移约束
参数输入说明	<code>stepName</code> (Step 的名称), <code>kbcCBCState</code> (位移约束)
参数输出说明	Bool 是否成功
API 实现	<code>virtual bool Insert(const QString&, const kbcCBCState&);</code>

3.3.15.3 kbcCConnDisplacementBCState

位移约束,关联在 **step** 下的相关参数和方法。包含成员变量 `mdlPropDouble corm1`, `corm2`, `corm3`, `corm4`, `corm5`, `corm6`，用于存储 6 个方向位移。

作用描述	位移约束构造函数
参数输入说明	无
参数输出说明	无
API 实现	<code>kbcCConnDisplacementBCState()</code>

3.3.16 Loads

3.3.16.1 kloCLoadContainer

存储 **model** 下的所有 **load** 对象，能够添加，删除 **load**。

3.3.16.1.1 UpdateSteps

作用描述	更新 Load 和 BC 的状态关系
参数输入说明	
参数输出说明	无
API 实现	<code>void UpdateSteps(const kprCStepNameProcedurePairList&, const QString&, bool suppress)</code>

3.3.16.1.2 Insert

作用描述	插入一个 step
参数输入说明	<code>step</code> 名称, <code>step</code> 对象,
参数输出说明	Bool 是否成功
API 实现	<code>bool Insert(const QString&, const kprCStep&)</code>

3.3.16.1.3 Remove

作用描述	删除指定的 step
参数输入说明	stepName (Step 的名称)
参数输出说明	Bool 是否成功
API 实现	<code>bool Remove(const QString&)</code>

3.3.16.2 kloCLoad

载荷类，用于保存各种载荷，成员变量 **rgnCRegion region**，存储载荷作用域；**QString fieldName** 保存离散场名称；

3.3.16.2.1 Insert

作用描述	载荷与载荷值关联
参数输入说明	stepName (Step 的名称)， kloCLoadState (位移约束)
参数输出说明	Bool 是否成功
API 实现	<code>bool Insert(const QString&, const kloCLoadState&);</code>

3.3.16.3 kloCLoadState

载荷值类，用于存储载荷大小。

3.3.16.4 KloCConcentratedForce

集中力载荷，包含其相关的参数及函数。包含成员变量 **DistribTypeEnum distribution**，用于判断载荷是否均匀分布，非均匀分布则需要创建离散场，均匀分布则使用 **kloCConcentratedForceState** 保存集中力分量。

3.3.16.5 kloCConcentratedForceState

集中力载荷，关联在 **step** 下的相关参数和方法。包含成员变量 **mdlPropDouble c1~c3**，保存集中力分量。

作用描述	集中力载荷值构造函数
参数输入说明	无
参数输出说明	无
API 实现	<code>kloCConcentratedForceState()</code>

3.3.16.6 kloCMoment

弯矩载荷，包含其相关的参数及函数，定义及使用方法和集中力载荷类似。

3.3.16.7 kloCMomentState

弯矩载荷值，包含其相关的参数及函数，定义及使用方法和集中力载荷值类似。

3.3.16.8 kloCPressure

压力载荷，包含其相关的参数及函数。包含成员变量 `PressureTypeEnum distType`，用于判断载荷是否均匀分布，非均匀分布则需要创建离散场，`QString fieldName` 保存离散场名称，均匀分布则使用 `kloCPressureState` 保存压力载荷值。

3.3.16.9 kloCPressureState

压力载荷，关联在 `step` 下的相关参数和方法。包含成员变量 `mdlPropDouble magnitude`，保存压力载荷值。

作用描述	压力载荷值构造函数
参数输入说明	无
参数输出说明	无
API 实现	<code>kloCPressureState()</code>

3.3.16.10 kloCGravity

重力载荷，关联在 `step` 下的相关参数和方法。

3.3.16.11 kloCGravityState

重力载荷值，关联在 `step` 下的相关参数和方法。包含成员变量 `mdlPropDouble c1~c3`，存储重力载荷值分量。

作用描述	重力载荷值构造函数
参数输入说明	无
参数输出说明	无
API 实现	<code>kloCGravityState()</code>

3.3.16.12 kloCLineLoad

线载荷，包含其相关的参数及函数。包含成员变量 `SystemTypeEnum system` 用于判断载荷的坐标系，包含单元局部坐标系和全局坐标系；`DistribTypeEnum distribution`，用于判断载荷是否均匀分布，非均匀分布则需要创建离散场，`QString fieldName` 保存离散场名称，均匀分布则使用 `kloCLineLoadState` 保存线载荷值。

3.3.16.13 kloCLineLoadState

线载荷，关联在 **step** 下的相关参数和方法。包含成员变量 `mdlPropDouble c1~c3`，保存线载荷值分量。

作用描述	压力载荷值构造函数
参数输入说明	无
参数输出说明	无
API 实现	<code>kloCPressureState()</code>

3.3.17 Loadcase

3.3.17.1 klcCLoadCaseCont

存储 **step** 下的所有 `klcCLoadCase` 对象，能够添加，删除 `klcCLoadCase`。

3.3.17.1.1 Insert

作用描述	插入工况
参数输入说明	<code>LoadCaseName</code> (工况的名称), <code>klcCLoadCase</code> (工况对象)
参数输出说明	Bool 是否成功
API 实现	<code>bool Insert(const QString&, const klcCLoadCase&);</code>

3.3.17.2 klcCLoadCase

工况类，包含成员变量 `clsxpMapString2Float boundaryConditions`，存储工况关联的位移/约束条件及系数；`clsxpMapString2Float loads`，用于存储工况关联载荷及系数。

3.3.17.2.1 boundaryConditions.Insert/loads.Insert

作用描述	在工况的约束/载荷列表中插入对象
参数输入说明	<code>BCName /loadName</code> (约束名称/载荷名称), <code>factor</code> (系数)
参数输出说明	Bool 是否成功
API 实现	<code>inline bool Insert(const QString&, const float&)</code>

3.3.18 Mesh

3.3.18.1 utiCoordCont3D

3.3.18.1.1 utiCoordCont3D

作用描述	构造节点坐标数据对象
参数输入说明	<code>initNumCoord</code> 节点个数

参数输出说明	无
API 实现	utiCoordCont3D(int initNumCoord)

3.3.18.2 bmeElementClass

3.3.18.2.1 ConstructObject

作用描述	构造单元类型类对象
参数输入说明	numElements 当前单元类型下单元个数，elTypLabel 当前单元类型，connectivity 当前单元类型下每个单元所对应的节点的 indexID
参数输出说明	无
API 实现	<pre>static bmeElementClass* ConstructObject(const int numElements, const QString& elTypLabel, int* connectivity)</pre>

3.3.18.2.2 SetUserElementLabel

作用描述	设置当前单元类型所对应单元的 label
参数输入说明	l 当前单元类型的 label 数组
参数输出说明	无
API 实现	void SetUserElementLabel(int* l)

3.3.18.3 ptsKMeshFactory

3.3.18.3.1 FromInputFile

作用描述	根据导入的文件对象创建网格
参数输入说明	part (ptoKPart 对象)，numNodes (节点个数)，nodeLabelArray (节点 label 数组)，coord (节点坐标数据对象)，numClasses (单元类型个数)，classes (单元类型数据)
参数输出说明	无
API 实现	<pre>static int FromInputFile(ptoKPart& part, uint numNodes, const cowListInt& nodeLabelArray, utiCoordCont3D& coord, uint numClasses, bmeElementClass** classes)</pre>

3.3.18.4 bmeMesh

3.3.18.4.1 mesConstGetMesh

作用描述	获取网格数据
参数输入说明	ftrFeatureList (特征列表)，instId (特征 ID)
参数输出说明	bmeMesh (网格数据对象)
API 实现	const bmeMesh* mesConstGetMesh(const ftrFeatureList& flist,

	<code>int instId)</code>
--	--------------------------

3.3.19 渲染更新

4 输入输出格式说明

- (1) 关键词行必须以*开始，注释行是以**开始；在参数之间用“,” 隔开。
- (2) 输入数据文件的名字为“ABE-xxx.in”，其中 xxx 为分析步的名字。
- (3) 输入数据中的内容采用英文（或拼音）表达。

4.1 输入格式关键字

4.1.1 *Heading

作用：一些信息描述，包含模型名称、文件生成时间等

格式：

- (1) 第一行：*Heading
- (2) 后续：使用注释加入描述语句

备注：

- (1) 保留关键字，可忽略

4.1.2 *Part

作用：声明一个 Part，包含节点、单元、节点集合和单元集合

格式：

- (1) 第一行：*Part, name=Part-1, Acoustic Field

其中：

name 参数指定 Part 的名称，不可省略

Acoustic Field 用于区分场点 Part，有该属性即为场点 Part

- (2) 中间行：为节点、单元等信息，遵循各自关键字格式

(3) 结束行:*End Part，必须以此表示 Part 信息结束

备注:

(1) 保留关键字，可忽略

4.1.3 *Node

作用: 声明节点信息

格式:

(1) 第一行: *Node

(2) 后续行: 节点编号, 节点 X 坐标, 节点 Y 坐标, 节点 Z 坐标

备注:

(1) 节点编号允许不连续, 但不可重复

(2) 节点信息一直到下一关键字出现结束, 中间可以有注释行, 如:

```
*Node
```

```
1, 0.0,0.0,0.0
```

```
** annotation
```

```
2,0.0,0.0,1.0
```

4.1.4 *Element

作用: 声明单元信息

格式:

(1) 第一行: *Element,type=ABS4,elset=Set-SHELL

其中:

type 参数指定单元类型, 不可省略

elset 参数指定单元集合名称, 为可选参数, 若指定则会将后续列出的所有单元放置在单元集合,

(2) 后续行: 单元编号, 节点 1 编号, 节点 2 编号, 节点 3 编号, 节点 4 编号

备注:

(1) 单元编号允许不连续, 但不可重复

(2) 单元信息一直到下一关键字出现结束, 中间可以有注释行, 如:

```
*Element,type=ABS4
```

```
1, 1,2,4,3
```

**** annotation**

2,5,6,8,7

(3) 单元节点顺序遵循左手螺旋定则

4.1.5 *Nset

作用：声明节点集合信息

格式：

(1) 第一行：*Nset, nset=Set-1,instance=Part-1-1

其中：

nset 参数指定节点集合名称，不可省略

instance 参数指定集合中节点所属的 instance

(2) 后续行：列出属于集合中的节点编号，以逗号隔开，每行不超过 16 个，且有后续行时以逗号结尾

备注：

(1) 在 Part 内部的 Nset 没有 instance 参数

(2) Nset 不可重名

4.1.6 *Elset

作用：声明单元集合信息

格式：

(1) 第一行：*Elset, elset=Set-1,instance=Part-1-1

其中：

elset 参数指定单元集合名称，不可省略

instance 参数指定集合中单元所属的 instance

(2) 后续行：列出属于集合中的单元编号，以逗号隔开，每行不超过 16 个，且有后续行时以逗号结尾

备注：

(1) 在 Part 内部的 Elset 没有 instance 参数

(2) Elset 不可重名

(3) Elset 可与 Nset 重名

4.1.7 *Chief

作用：定义 Chief 方法的 Chief 点坐标

格式：

- (1) 第一行：*Chief
- (2) 第二行：1, 1.0, 0.0, 0.0
编号, X 坐标, Y 坐标, Z 坐标

4.1.8 *Assembly

作用：声明装配体信息，包含实例、节点集合和单元集合。

格式：

- (1) 第一行：*Assembly, name=assembly
其中：
name 参数指定 Assembly 名称，不可省略
- (2) 结尾行：*End Assembly

备注：

- (1) 保留关键字，可忽略

4.1.9 *Instance

作用：声明实例信息

格式：

- (1) 第一行：*Instance, name=part-1-1, part=part-1
其中：
name 指定 Instance 名称，不可省略
part 指定使用的 Part 名称，不可省略
- (2) 结尾行：*End Instance

备注：

- (1) 保留关键字，可忽略
- (2) Instance 不可重名
- (3) 多个 Instance 可以指向同一个 Part

4.1.10 *Submodel

作用：指定子模型对象

描述：用于结构网格向声学网格传递数据，一般是指结构网格。

格式：

(1) 第一行 *Submodel, type=NODE, absolute exterior tolerance=0.01, global elset=Assembly,CAB-OUTSID

其中：

absolute exterior tolerance：指定容差值

global elset：数据来源单元集合

CAB-OUTSID：对应需要转移数据来源单元集合的对象名称

(2) EXTERIOR.AIROUT-ASI-NODES,

其中：

AIROUT-ASI-NODES：指定数据转移目标节点集合

4.1.11 *Beam Section

作用：指定梁单元属性

格式：

设置类型 **Section integration** : **During analysis**

(1) 第一行 : *Beam Section, elset=Set-7, material=Material-2, temperature=GRADIENTS, section=L

其中：

elset 参数指定单元集合，该集合内的所有单元皆使用同样属性，不可省略。

material 参数指定梁单元材料，不可省略。

temperature 参数指温度属性。

seciton 参数指的是梁的型材类型。

- (2) 第二行：梁对应的截面参数
- (3) 第三行：梁对应的截面方向
- (4) 第四行：积分点个数
- (5) 第五行：梁对应的 OffSet1 参数
- (6) 第六行：梁对应的 OffSet2 参数

设置类型 Section integration : Before analysis

- (1) 第一行：*Beam General Section, elset=Set-1, section=GENERAL

其中：

elset 参数指定单元集合，该集合内的所有单元皆使用同样属性，不可省略

seciton 参数指的是梁的型材类型。

- (2) 第二行：梁对应的截面参数
- (3) 第三行：梁对应的截面方向
- (4) 第四行：梁对应的材料属性
杨氏模量，剪切模量
- (5) 第五行：梁对应的 OffSet1 参数
- (6) 第六行：梁对应的 OffSet2 参数

备注：

- (1) 保留关键字，可忽略

4.1.12 *Shell Section

作用：指定壳单元属性

格式：

- (7) 第一行：*Shell Section, elset=Set-Section, material=Material-1

其中：

elset 参数指定单元集合，该集合内的所有单元皆使用同样属性，不可省略
material 参数指定壳单元材料

(8) 第二行：壳厚度，壳厚度方向积分点个数

备注：

(2) 保留关键字，可忽略

4.1.13 *Material

作用：定义材料属性

格式：

(1) 第一行：*Material name=Material-1

其中：

name 指定材料名称，不可忽略

(2) 后续行：由各材料相关关键字自由组合

备注：

(1) 此关键字需与其它材料相关关键字配合使用

4.1.14 *Density

作用：定义材料密度

格式：

(1) 第一行：*Density

(2) 第二行：密度值

备注：

(1) 此关键字需与*Material 关键字配合使用

4.1.15 *Acoustic Medium

作用：定义材料声学数据

格式：

(1) 第一行：*Acoustic Medium

(2) 第二行：相关参数

备注：

-
- (1) 此关键字需与*Material 关键字配合使用

4.1.16 *Amplitude

作用：定义频率相关物理量变化曲线，可用于载荷和边界条件

格式：

- (1) 第一行：*Amplitude,name=Amp-1

其中：

name 参数指定名称，不可省略

- (2) 后续行：频率值 1，幅值 1，频率值 2，幅值 2

备注：

- (1) 不可重名
- (2) 每一行不超过两组数据
- (3) 每一行结尾没有逗号
- (4) 线性插值

4.1.17 *Step

作用：定义声学计算分析步的相关参数，包括问题类型、计算频率、边界条件等

格式：

- (1) 第一行：*Step, name=Step-1

其中：

name 参数指定分析步名称，不可省略

- (2) 第二行：*Acoustic BEM, Radiation, Inner

其中：

Acoustic BEM: 表示声学边界元

Inner 表示直接内部声场计算、Outer 表示直接边界外部声场计算、Bilateral 表示间接边界元声场计算

Radiation: 辐射 Scattering:散射

- (3) 第三行：起始频率，终止频率，频率点个数
- (4) 后续行：继续定义计算频率或者由各分析步相关关键字配合使用
- (5) 结束行：*End Step

备注：

- (1) 不可重名
- (2) 每行只能定义一组计算频率
- (3) 某些求解器需要设置的参数

4.1.18 *Boundary

作用：定义边界条件

格式一，用户定义边界条件：

- (1) 第一行：*Boundary Condition
- (2) *Boundary, name=BC-3
- (3) 第二行：set-1,1,1.000,0.000

第一列 set-1 参数表示施加边界条件的单元/节点集合

第二列数据表示边界条件：声压 0，法向速度 1，声阻抗 2

第三列和第四列用于表示边界条件值的实部或虚部，不可省略

格式二，频变边界条件：

- (1) 第一行：* Boundary Condition
 - (2) 第二行：* Boundary,amplitude=Amp-1,load case=1
- Amplitude 参数指定名称，不可省略
- load case 参数值 1 定义实部，2 表示虚部
- (3) 第三行：Set-1,1,1.0

第一列表示施加位置的集合

第二列数据表示边界条件：声压 0，法向速度 1，声阻抗 2

第三列用于表示边界条件幅值比例

格式三，结构振动数据导入（可以是速度，也可以是声压）：

- (1) *Boundary, submodel, step=1
- Submodel：标志，表明边界数据来自子模型。
- step：数据来源子模型分析步序号

- (2) EXTERIOR.AIROUT-ASI-NODES, 1, 1

EXTERIOR：instance 对象名称

AIROUT-ASI-NODES：指定数据转移目标节点集合名称

1,1 指定第一个自由度编号和最后一个自由度的编号，（1~6 为平移旋转，8 为声

压)

备注:

- (1) 不可重名
- (2) 一个分析步内不同边界条件不可施加在相同的单元上

4.1.19 *ALoad

作用: 定义载荷

格式:

- (1) 第一行: *ALoad
- (2) 第二行: 1,1.000,1.000,0.000,0.000

第一列 声源类型 0 入射声波, 1 点声源, 2 平面波

第二列 幅值

第三列到第五列 点源位置/平面波方向

5 输出格式说明

HDF5 文件一般以.h5 或.hdf5 作为后缀,需要专门的软件才能打开文件以查看内容。HDF5 文件结构为树状结构,主要分为两类: **Group** (组) 和 **Datasets** (数据块), **Group** 类似文件夹,用来组织数据; **Datasets** 类似于二维数组,存放关键数据。具体数据结构请参看 HDF 官网 <https://www.hdfgroup.org>。

5.1 模型信息

5.1.1 Group: Parts

作用: 作为根目录,包含模型所有的 Part。

内容:

(3) Group Name: Parts

目录名字固定为“Parts”

5.1.1.1 Group: Part

作用: 作为 Parts 的子目录,存放一个 Part 的信息,包含节点和单元信息。

内容:

(4) Group Name: Part Name

以 Part 的名字作为目录的名字,例如: Part-1、Part-2……

5.1.1.1.1 Group: Nodes

作用: 作为 Part 的子目录,存放节点信息

内容:

(3) Group Name: Nodes

数据块名字固定为“Nodes”

5.1.1.1.1.1 DataSet: Labels

作用: Nodes 下的数据块,存放当前 Part 下节点的标签

内容:

(1) Dataset Name: Labels

数据块名字固定为“Labels”

(2) 此数据块为 $N \times 1$ 的数组，每行存放一个节点的标签。

5.1.1.1.1.2 DataSet: Coordinates

作用: Nodes 下的数据块，存放当前 Part 下节点的坐标信息

内容:

(1) Dataset Name: Coordinates

数据块名字固定为“Coordinates”

(2) 此数据块为 $N \times 3$ 的数组，每行存放一个节点的三维坐标。

5.1.1.1.2 Group: Elements

作用: 作为 Part 的子目录，存放单元信息

内容:

(1) Group Name: Elements

数据块名字固定为“Elements”

5.1.1.1.2.1 Group: Element Class

作用: 作为 Elements Type 的子目录，存放 Element Class 信息

内容:

(1) Group Name: Element Class

以 Element Class 的编号作为目录的名字，例如: ElementClass:1

(5) Group 属性:

[1] ElementType: Type 为 String，长度 length 为 Value 字符串的长度，Array Size 为 1，Value 为单元类型名字，例如: S4R

[2] SectionCategory: Type 为 String，长度 length 为 Value 字符串的长度，Array Size 为 1，Value 为截面类型名字，例如: shell

Dataset: labels

作用: Element Class 下的数据块，声明 Element Class 中的单元标签

内容:

(3) Dataset Name: labels

数据块名字固定为“labels”

- (4) 此数据块为 $N \times 1$ 的数组，每行存放一个单元的标签。
- (5) 此数据块的行数为当前 Element Class 下的单元数量。

Dataset: connectivity

作用：Element Class 下的数据块，声明 Element Class 中每个单元包含的节点的标签

内容：

- (1) **Dataset Name: connectivity**
数据块名字固定为 “connectivity”
- (2) 此数据块每行存放一个单元包含的所有节点的标签，包含节点的数量根据 Element Class 所属的 Element Type 来确定。
- (3) 此数据块的行数为当前 Element Class 下的单元数量。

5.1.2 Group: Assembly

作用：作为根目录，存放装配体信息。

内容：

- (3) **Group Name: Assembly**
目录名字固定为 “Assembly”

5.1.2.1 Group: Instances

作用：作为 Assembly 的子目录，包含所有的实例。

内容：

- (3) **Group Name: Instances**
目录名字固定为 “Instances”

5.1.2.1.1 Group: Instance

作用：作为 Instances 的子目录，存放一个实例的信息。

内容：

- (1) **Group Name: Instance Name**
以 Instance 的名字作为目录的名字
- (2) **Group 属性：**
 - [1] **Dependent:** 属性为整型，值为 1 表示依赖 part，值为 0 表示不依赖
 - [2] **Part Name:** Type 为 String，长度 length 为 Value 字符串的长度，Array Size

为 1，Value 为依赖的 Part 的名称

5.1.2.1.1.1 Group: NodeSets

作用：作为 Instance 的子目录，包含当前部件下的所有节点集。

内容：

- (1) Group Name: NodeSets

目录名字固定为“NodeSets”

Group: NodeSet

作用：作为 NodeSets 的子目录，存放一个节点集的信息。

内容：

- (1) Group Name: Node Set Name

以节点集的名字作为目录的名字

DataSet: subset data

作用：作为 NodeSet 的子目录，存放一个子节点集的信息。

内容：

- (1) Dataset Name: subset index

数据块名字为子节点集的索引值

- (2) 行数据：每行表示一个节点的标签

- (3) 列数据：排列全部节点

5.1.2.1.1.2 Group: ElementSets

作用：作为 Instance 的子目录，包含当前部件下的所有单元集。

内容：

- (1) Group Name: ElementSets

目录名字固定为“ElementSets”

Group: ElementSet

作用：作为 ElementSets 的子目录，存放一个单元集的信息。

内容：

- (1) Group Name: Element Set Name

以单元集的名字作为目录的名字

DataSet: subset data

作用：作为 ElementSet 的子目录，存放一个子单元集的信息。

内容：

(1) **Dataset Name:** subset index

数据块名字为子单元集的索引值

(2) **行数据：**每行表示一个单元的标签

(3) **列数据：**排列全部单元

(4) **Dataset 属性：**

[1] **ClassIndex:** Type 为 int, Array Size 为 1, Value 为子单元集所属的单元类型索引

5.1.2.2 Group: NodeSets

作用：作为 Assembly 的子目录，包含所有 Assembly 下的节点集。

内容：

(1) **Group Name:** NodeSets

目录名字固定为“NodeSets”

5.1.2.2.1 Group: NodeSet

作用：作为 NodeSets 的子目录，存放一个节点集的信息。

内容：

(1) **Group Name:** Node Set Name

以节点集的名字作为目录的名字

5.1.2.2.1.1 DataSet: subset data

作用：作为 NodeSet 的子目录，存放一个子节点集的信息。

内容：

(1) **Dataset Name:** subset index

数据块名字为子节点集的索引值

(2) **行数据：**每行表示一个节点的标签

(3) **列数据：**排列全部节点

5.1.2.3 Group: ElementSets

作用：作为 Assembly 的子目录，包含所有 Assembly 下的单元集。

内容：

- (1) Group Name: ElementSets

目录名字固定为“ElementSets”

5.1.2.3.1 Group: ElementSet

作用：作为 ElementSets 的子目录，存放一个单元集的信息。

内容：

- (1) Group Name: Element Set Name

以单元集的名字作为目录的名字

5.1.2.3.1.1 DataSet: subset data

作用：作为 ElementSet 的子目录，存放一个子单元集的信息。

内容：

- (1) Dataset Name: subset index

数据块名字为子单元集的索引值

- (2) 行数据：每行表示一个单元的标签

- (3) 列数据：排列全部单元

- (4) Dataset 属性：

[1] ClassIndex: Type 为 int, Array Size 为 1, Value 为子单元集所属的单元类型索引

5.2 结果信息

5.2.1 Group: Steps

作用：作为根目录，包含模型所有的 Step 信息。

内容：

- (1) Group Name: Steps

目录名字固定为“Steps”

5.2.1.1 Group: Step

作用：作为 Steps 的子目录，存放一个 Step 的信息。

内容：

(1) Group Name: Step Name

以 Step 的名字作为目录的名字，例如：Step-1、Step-2……

(2) Group 属性：

[1] Description: Type 为 String，长度 length 为 Value 字符串的长度，Array Size 为 1，Value 为 Step 的描述

[2] Domain: Type 为 int，Array Size 为 1，Value 为 Step 的域，**Value** 边界元的值固定为 2

5.2.1.1.1 Group: Frames

作用：作为子目录，存放当前 Step 下的所有 Frame。

内容：

(1) Group Name: Frames

目录名字固定为“Frames”

5.2.1.1.1.1 Group: Frame

作用：作为 Frames 的子目录，存放一个 Frame 的信息，包含当前步下的 Field Outputs 变量信息。

内容：

(1) Group Name: Frame Name

以 Frame 的名字作为目录的名字，例如：Frame0、Frame1、Frame2……

(2) Group 属性：

[1] Description: Type 为 String，长度 length 为 Value 字符串的长度，Array Size 为 1，Value 为 Frame 的描述

[2] Inc/Mode: Type 为 int，Array Size 为 1，Value 为 Frame 的增量值，仅在时域中输出

[3] Time/Freq: Type 为 float，Array Size 为 1，Value 为 Frame 的频率值，仅在频域中输出

Group: Field Output

作用：作为 Field Outputs 的子目录，存放一个 Field Output 变量的信息，包含变量属性描述、标志及实部和虚部数据集。

内容：

(1) Group Name: Field Output Name

以 Field Output 的名字作为目录的名字，例如：POR、U、UR……

(2) Group 属性：

[1] ComponentLabels: Type 为 String，长度 length 为单个分量字符串长度，Array Size 为分量数量，Value 为分量值，如 “U1, U2, U3”

[2] Invariant: Type 为 int，Array Size 为不变量的数量，Value 为不变量的值，如 “9, 10, 11, 1, 2, 3”

[3] Position: Type 为 int，Array Size 为 1，Value 值为 1 表示附着在节点上，值为 3 表示附着在积分点上，值为 4 表示附着在单元上

[4] Type: Type 为 int，Array Size 为 1，Value 值表示变量类型，2 表示 SCALAR 线性值，3 表示 VECTOR 矢量

Group: Field Output Instances

作用：作为 Field Output 的子目录，存放当前变量所属的实例。

内容：

(1) Group Name: Field Output Instances

固定名字 “Field Output Instances”

Group: Instance

作用：作为 Field Output Instances 的子目录，存放当前变量所属的实例。

内容：

(1) Group Name: Instance Name

以 Instance 的名字作为目录的名字

DataSet: Real

作用：Field Output Instance 下的数据块，声明实部数据

内容：

(1) Dataset Name: Real

数据块名字固定为 “Real”

(2) 行数据：每行表示一个节点的实部数据集

(3) 列数据：排列全部节点

备注：本文档仅包含节点上数据

DataSet: Img

作用：Field Output Instance 下的数据块，声明虚部数据

内容：

(1) Dataset Name: Img

数据块名字固定为 “Img”

(2) 行数据：每行表示一个节点的虚部数据集

(3) 列数据：排列全部节点

备注：本文档仅包含节点上数据